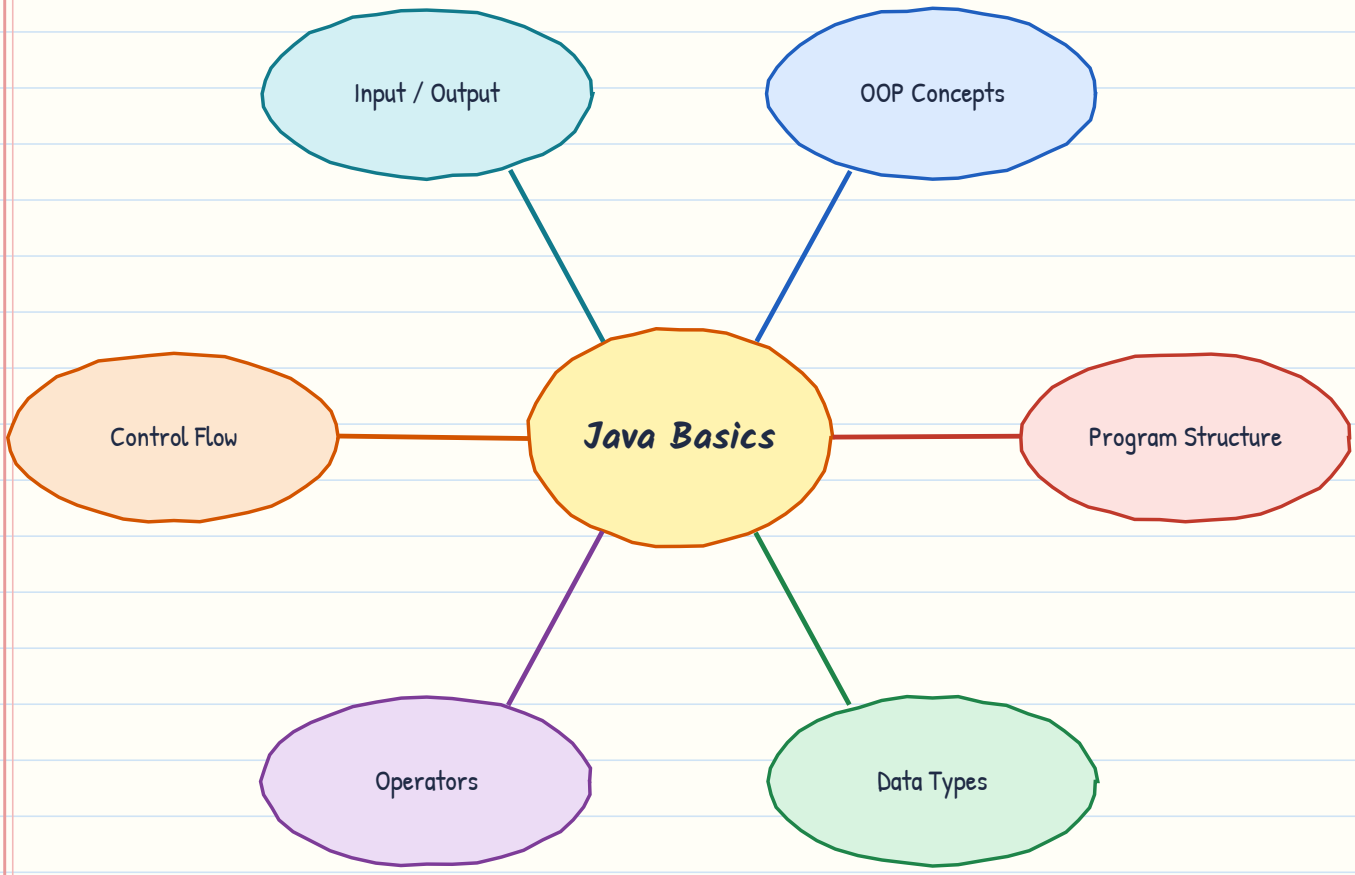
UNIT 1

OOP Basics, Data Types, Operators, Control

Program structure | Variables | Operators | Decisions & Loops



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. What is Object Oriented Programming?

OOP is a way of writing programs where we model real-world things as **objects**. Each object keeps its own ***data*** (state) and its own ***actions*** (behaviour) together.

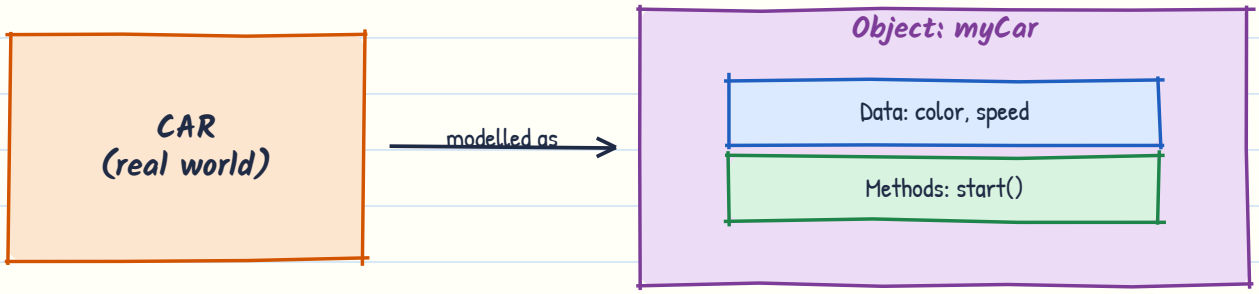


Fig 1: A real-world object -> a software object

Basic terms

Term	Meaning	Example
Class	Blueprint / template of objects	class Car
Object	Real instance made from a class	Car myCar = new Car();
Method	A function inside a class (behaviour)	start()
Message passing	Objects talk by calling methods	myCar.start()

The 4 Principles (pillars) of OOP

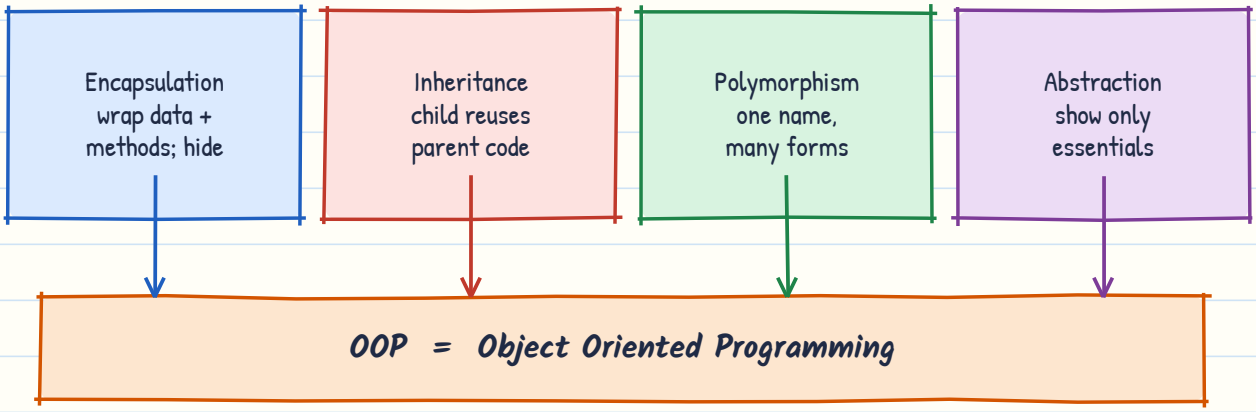


Fig 2: Four pillars of OOP

- ***Encapsulation*** - binding data and code in one unit (class); private data = safe data.
- ***Inheritance*** - a new class acquires properties of an existing class (code reuse).
- ***Polymorphism*** - same method name behaves differently (overloading / overriding).
- ***Abstraction*** - hide the "how", show only the "what" (abstract class, interface).

Memory trick

Easy memory trick: ATM = Abstraction hides the machinery, Encapsulation is the machine body, Polymorphism: "withdraw" works for savings or current, Inheritance: SavingsAccount IS-A Account.

2. Program Structure in Java

Every Java program is built from the parts shown below. Java source is saved as `ClassName.java`, compiled to ***bytecode*** (`.class`) and run on the ***JVM***.

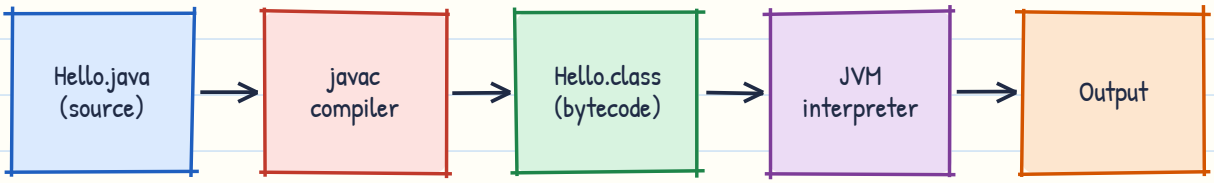


Fig 3: How a Java program runs (Write Once, Run Anywhere)

Structure of a simple program

Hello.java

```
import java.util.*;                  // 1. package (import) section
public class Hello {                 // 2. class declaration
    public static void main(String[] args) {   // 3. main method
        System.out.println("Hello, Java!");    // 4. statement
    }
}
```

```
> OUTPUT
Hello, Java!
```

- `public` - accessible to JVM from anywhere
- `static` - JVM calls main without creating an object
- `void` - main returns nothing
- `String[] args` - command line arguments array

Compile and run

```
javac Hello.java      // creates Hello.class
java Hello            // runs the program (no .class!)
```

Rule

The file name must match the public class name (case sensitive). Hello.java -> public class Hello.

Elements (Tokens) of a Java program

A *token* is the smallest meaningful unit the compiler reads. Java has five kinds:

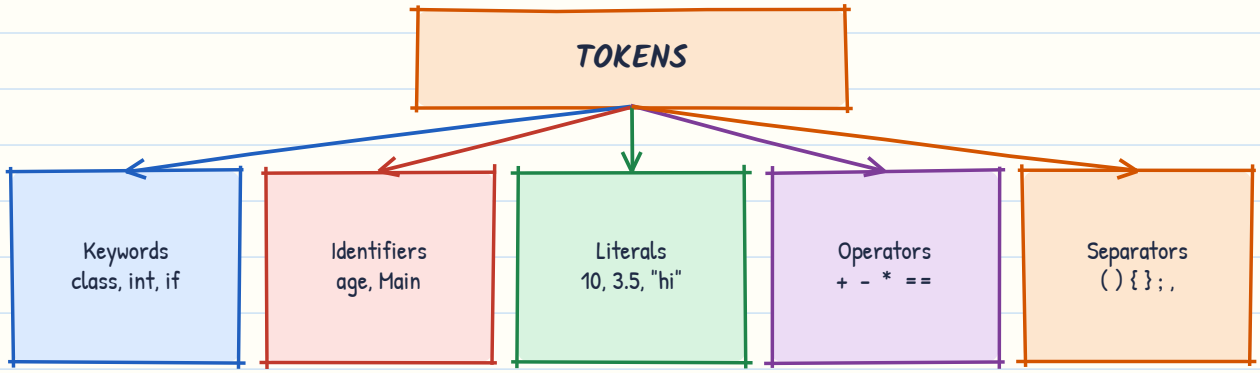


Fig 4: Java tokens

- *Identifier rules:* letters, digits, _, $; cannot start with a digit; no keywords; case-sensitive.
  - valid: `total`, `_x`, `$cost` | invalid: `2sum`, `my-var`, `class`
- Java has 50+ *reserved keywords* (they have fixed meaning).

Java statements

Type	Description	Example
Declaration	creates a variable	int a;
Expression	calculation / assignment	a = 5 + 2;
Control	changes flow of execution	if, for, while
Empty / Block	nothing / group inside { }	; or { ... }

Command Line Arguments

Values typed after the class name when running are stored as Strings in `args[]`.

Add.java *run: java Add 10 20*

```java
public class Add {
    public static void main(String[] args) {
        int a = Integer.parseInt(args[0]);   // text -> int
        int b = Integer.parseInt(args[1]);
        System.out.println("Sum = " + (a + b));
    }
}
```

```
> OUTPUT
Sum = 30
```

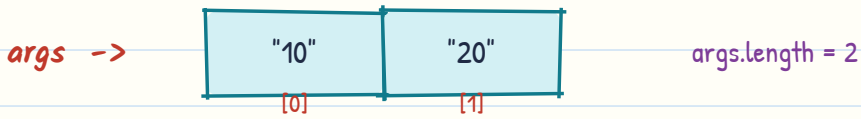


Fig 5: args[] after java Add 10 20

User input using Scanner

Input.java

```java
import java.util.Scanner;
public class Input {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        System.out.println(name + " is " + age);
        sc.close();
    }
}
```

Method	Reads	Method	Reads
nextInt()	int	nextDouble()	double
next()	one word	nextLine()	whole line
nextBoolean()	boolean	nextLong()	long

Escape sequences & comments

Escape	Meaning	Escape	Meaning
\n	new line	\t	tab space
\"	double quote	\\	backslash
\'	single quote	\r	carriage return

```
// single line comment
/* multi
   line comment */
/** documentation comment (javadoc) */
```

Good style

Programming style: meaningful names, camelCase for variables/methods (totalMarks), PascalCase for classes (StudentRecord), UPPER_CASE for constants, indent with 4 spaces, one statement per line.

3. Data Types, Variables & Operators

Java is ***strongly typed***: every variable must have a declared type. Types are divided into two families.

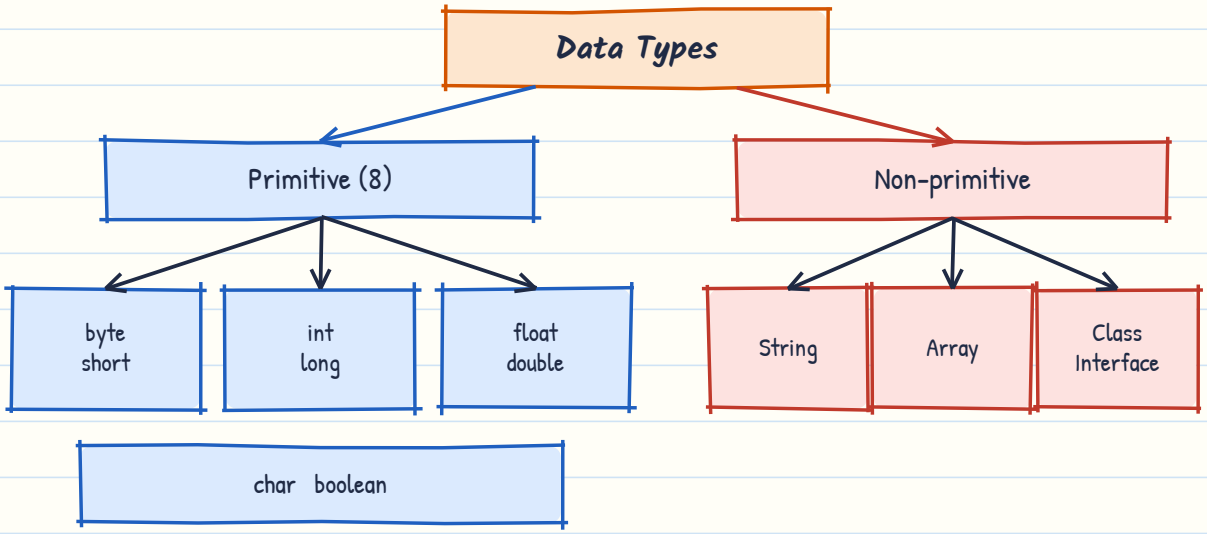


Fig 6: Classification of Java data types

Type	Size	Range / Use	Default
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0
int	4 bytes	about -2.1 to +2.1 billion	0
long	8 bytes	very large whole numbers (123L)	0L

float	4 bytes	decimal, 6-7 digits (3.5f)	0.0f
double	8 bytes	decimal, 15 digits (default)	0.0
char	2 bytes	single Unicode char ('A')	\u0000
boolean	1 bit*	true / false	false

Declaring variables

```
int age;                    // declaration
age = 20;                   // assignment
double price = 99.5;        // declaration + initialization
int a = 5, b = 10, c;       // many at once
final double PI = 3.14159;   // constant (cannot change)
```

Type casting

Converting one data type into another. ***Widening*** is automatic and safe; ***narrowing*** needs a manual cast and may lose data.

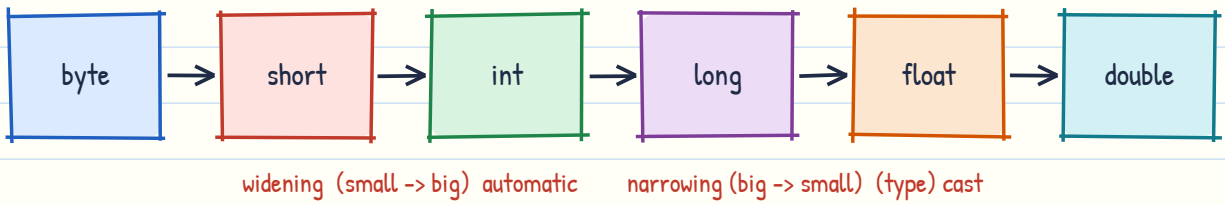


Fig 7: Widening (auto) -> and Narrowing (manual) <-

```
int i = 100;
double d = i;             // widening: 100.0
double pi = 9.78;
int x = (int) pi;         // narrowing: 9  (decimal lost)
char ch = (char) 66;      // 'B'
```

Scope of a variable

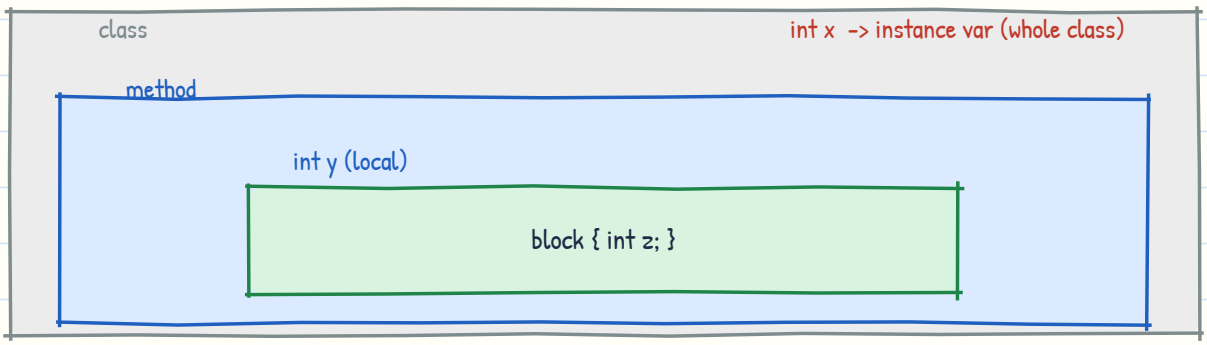


Fig 8: Variable scope - where a variable is alive

- ***Instance variable*** - declared in class, one copy per object.

- ***Local variable*** – declared inside method/block; dies when block ends; no default value.
- ***Static variable*** – one shared copy for the whole class.

Literals, constants & printf()

Literal constant = fixed value written in code: `10`, `3.14`, `'A'`, `"Java"`, `true`. ***Symbolic constant*** = named constant using `final`: `final int MAX = 100;`

```
System.out.printf("Name: %s, Age: %d%n", "Ravi", 21);
System.out.printf("Price: %.2f%n", 99.5);
System.out.printf("[%5d] [%-5d]%n", 42, 42);
```

```
> OUTPUT
Name: Ravi, Age: 21
Price: 99.50
[   42] [42   ]
```

Format	Prints	Format	Prints
%d	int	%f	decimal
%s	String	%c	char
%.2f	2 decimals	%n	new line
%b	boolean	%x	hex

static and final

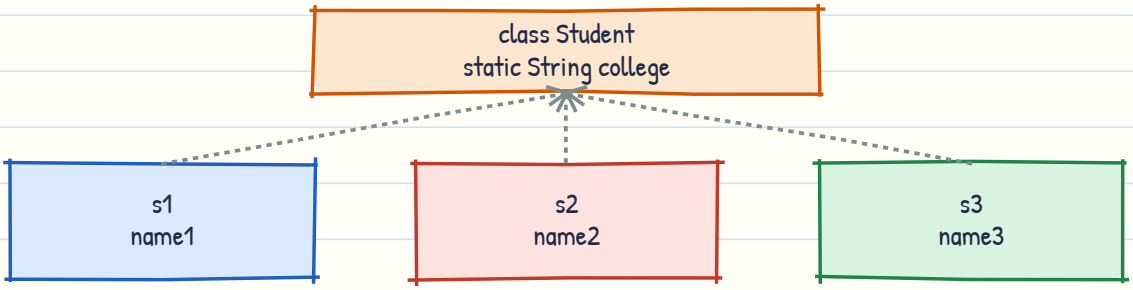


Fig 9: static variable is shared by all objects

- `static` variable/method belongs to the ***class***; call as `ClassName.method()`; static methods cannot use `this` or non-static members directly.
- `final` variable = constant; `final` method = cannot be overridden; `final` class = cannot be inherited.

Operators

Category	Operators	Example
Arithmetic	+ - * / %	a % b -> remainder
Assignment	= += -= *= /=	x += 5 (x = x+5)
Increment / Decrement	++ --	i++ ++i
Relational	!= > < >=	a > b

Logical	&& || !	(a>1) && (b<5)
Bitwise	& | ^ ~ << >> >>>	5 & 3 = 1
Ternary	cond ? x : y	max = a>b ? a : b

j = i++
use first (j=5)
then i becomes 6

j = ++i
increase first (i=6)
then j = 6

Fig 10: Pre vs post increment, int i = 5

```
int a = 10, b = 3;
System.out.println(a / b);      // 3     (int division)
System.out.println(a % b);      // 1
System.out.println(a > b && b > 0);   // true
System.out.println(5 & 3);      // 1   (101 & 011 = 001)
System.out.println(5 | 3);      // 7
System.out.println(5 ^ 3);      // 6
int max = (a > b) ? a : b;      // 10
```

Precedence & associativity (high -> low)

Level	Operators	Associativity
1	() [] .	left to right
2	++ -- ! ~ (unary)	right to left
3	* / %	left to right
4	+ -	left to right
5	<< >>	left to right
6	< <= > >=	left to right
7		left to right
8	& ^ |	left to right
9	&& then ||	left to right
10	?:	right to left
11	= += -=	right to left

Tip

BODMAS-like: brackets first, then * / %, then + -. When confused, just add ( ) !

4. Control Statements

Control statements decide **which** code runs and **how many times**. Two families: selection (decisions) and iteration (loops), plus jump statements.

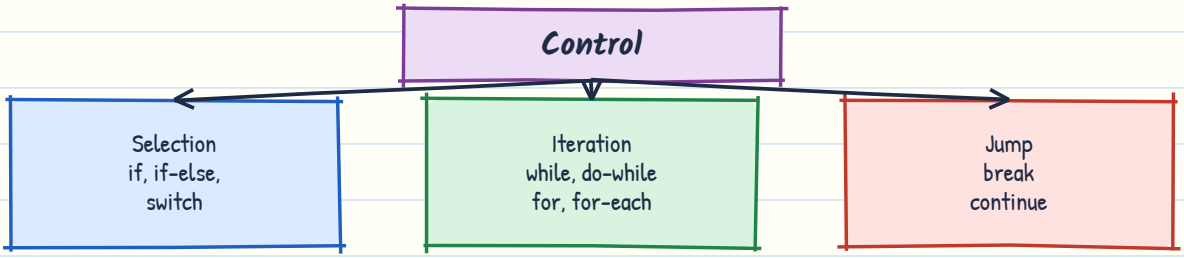


Fig 11: Control statements

if, if-else, nested if

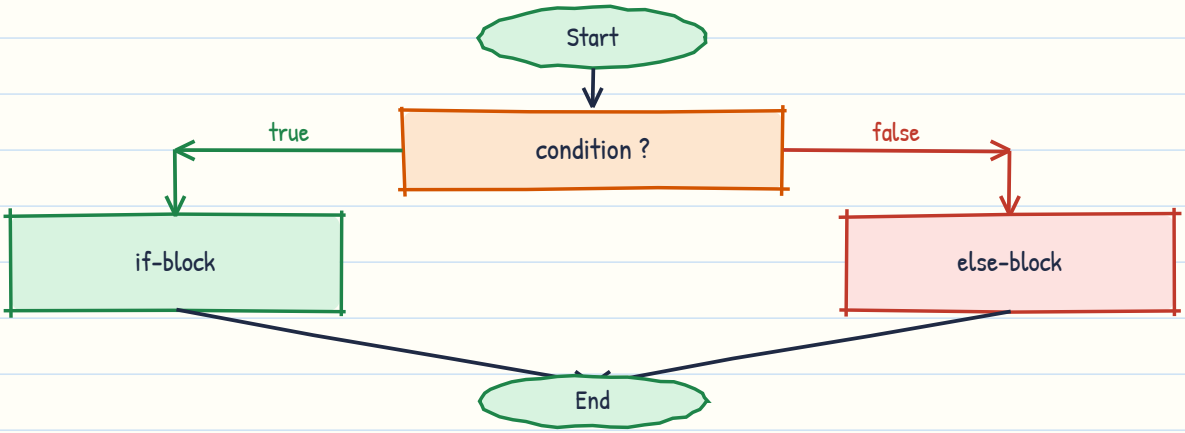


Fig 12: Flowchart of if-else

```java
int marks = 72;
if (marks >= 90)       System.out.println("Grade A");
else if (marks >= 60)  System.out.println("Grade B");   // runs
else                   System.out.println("Fail");
```

```
> OUTPUT
Grade B
```

Nested if: an `if` inside another `if` - inner one is tested only when the outer is true.

```java
if (age >= 18) {
    if (hasId)  System.out.println("Allowed");
    else        System.out.println("Bring ID");
}
```

switch statement

```
int day = 3;
switch (day) {
    case 1:  System.out.println("Mon"); break;
    case 2:  System.out.println("Tue"); break;
    case 3:  System.out.println("Wed"); break;    // runs
    default: System.out.println("Other");
}
```

Common mistake

Forget `break` and execution FALLS THROUGH into the next case! switch works with int, char, String, enum (not long/float/double).

Iteration statements

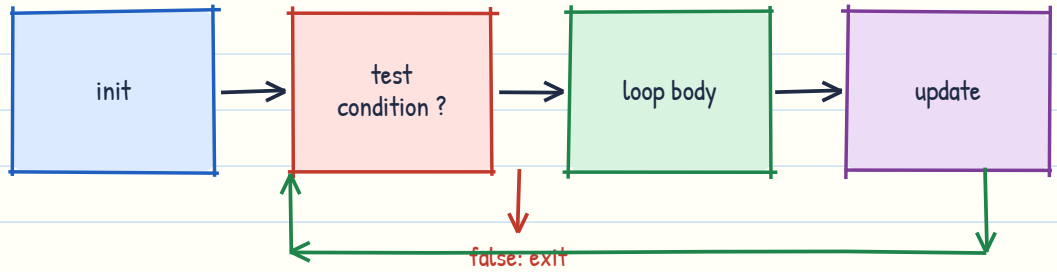


Fig 13: Loop flow (entry-controlled: while / for)

Loop	Type	Use when
while	entry-controlled	number of repeats unknown
do-while	exit-controlled	body must run at least once
for	entry-controlled	number of repeats known
for-each	collection loop	visit every array element

```
int i = 1;
while (i <= 3) { System.out.print(i + " "); i++; }     // 1 2 3
int j = 10;
do { System.out.println("runs once"); } while (j < 5);
for (int k = 1; k <= 5; k++) System.out.print(k + " ");  // 1 2 3 4 5
int[] a = {10, 20, 30};
for (int v : a) System.out.print(v + " ");           // 10 20 30
```

Nested for loop - pattern

```java
for (int i = 1; i <= 4; i++) {          // rows
    for (int j = 1; j <= i; j++)       // columns
        System.out.print("* ");
    System.out.println();
}
```

```
> OUTPUT
*
* *
* * *
* * * *
```

break and continue

break;	continue;
exit the loop completely	skip rest of body, next iteration

Fig 14: break leaves the loop, continue skips one round

```java
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;      // skips 3
    if (i == 5) break;         // stops at 5
    System.out.print(i + " ");
}
```

```
> OUTPUT
1 2 4
```

Quick Revision & Exam Questions

Topic	One-line memory
OOP pillars	Encapsulation, Inheritance, Polymorphism, Abstraction
Execution	.java -> javac -> .class -> JVM
Tokens	Keyword, Identifier, Literal, Operator, Separator
Casting	Widening auto, narrowing manual (type)
static	belongs to class, one shared copy
final	variable=constant, method=no override, class=no inherit
switch	needs break, else fall-through
Loops	while/for check first, do-while checks last

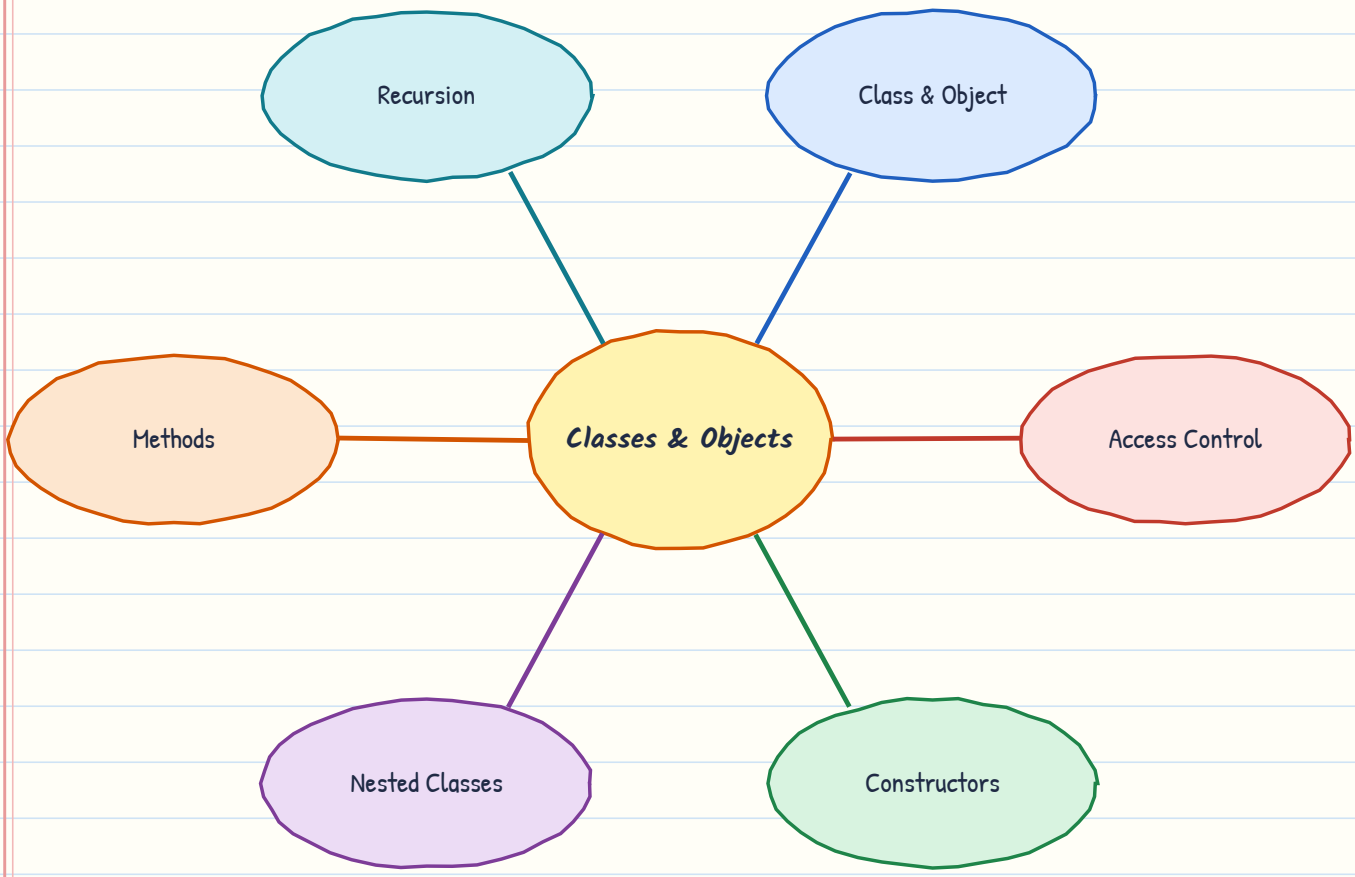
Likely exam questions

- Explain the basic concepts and principles of OOP with examples.
- Describe the structure of a Java program; explain compile and run steps.
- Differentiate primitive and non-primitive data types; explain type casting.
- Explain static and final with examples.
- Explain operator precedence; write a note on bitwise operators.
- Differentiate while, do-while and for; explain break and continue.
- Write a program to read a number with Scanner and print its factorial.

UNIT II

Classes and Objects, Methods

Declaration | Constructors | Access control | Overloading | Recursion



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

I. Classes and Objects

A ***class*** is a user-defined blueprint that groups data (fields) and code (methods). An ***object*** is a real, memory-occupying instance of that class.

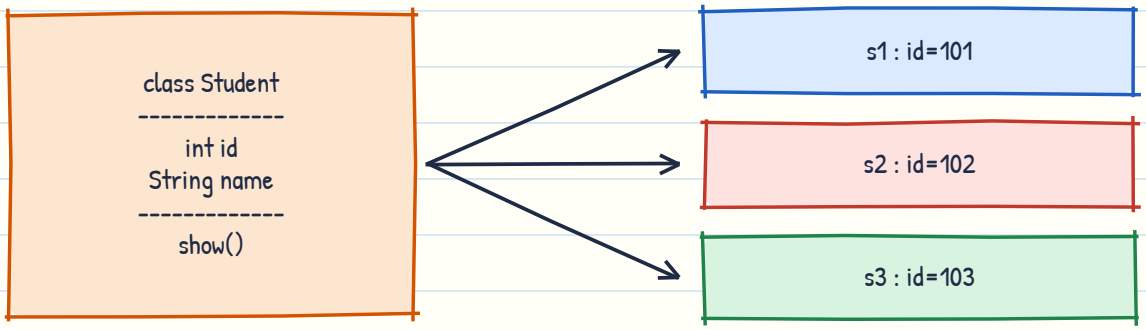


Fig 1: One class -> many objects (like one cake mould -> many cakes)

Class declaration & modifiers

```
[modifier] class ClassName {
    // fields (variables)
    // constructors
    // methods
}
```

Modifier	Meaning
public	class visible to every other class
(default)	visible only inside same package
abstract	cannot create objects; meant to be extended
final	cannot be inherited (no subclass)

Class members & creating objects

Student.java

```java
class Student {
    int id;  String name;            // instance variables
    void show() {                    // instance method
        System.out.println(id + " " + name);
    }
    public static void main(String[] args) {
        Student s1 = new Student();  // create object
        s1.id = 101;  s1.name = "Asha";
        s1.show();                   // call method
    }
}
```

```
> OUTPUT
101 Asha
```

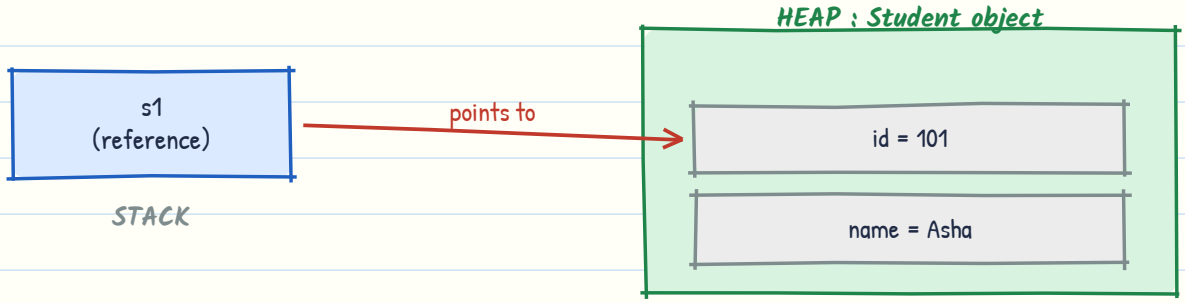


Fig 2: Student s1 = new Student(); - reference vs object in memory

Key idea

new allocates memory in the HEAP and returns its address. The variable s1 only stores that address (a reference), not the object itself.

Assigning one object to another

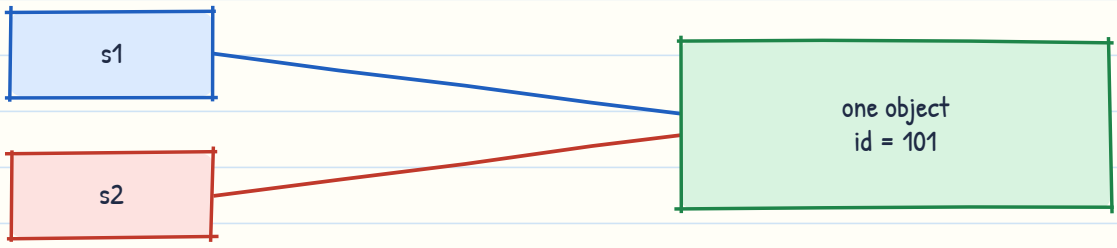


Fig 3: s2 = s1 copies the REFERENCE, not the object

```java
Student s1 = new Student();  s1.id = 1;
Student s2 = s1;             // both point to same object
s2.id = 99;
System.out.println(s1.id);   // 99  (changed through s2!)
```

```
> OUTPUT
99
```

2. Access Control

Access modifiers decide *who can see* a member. Remember the order: private < default < protected < public.

Modifier	Same class	Same package	Subclass (other pkg)	Anywhere
private	YES	no	no	no
default	YES	YES	no	no
protected	YES	YES	YES	no
public	YES	YES	YES	YES

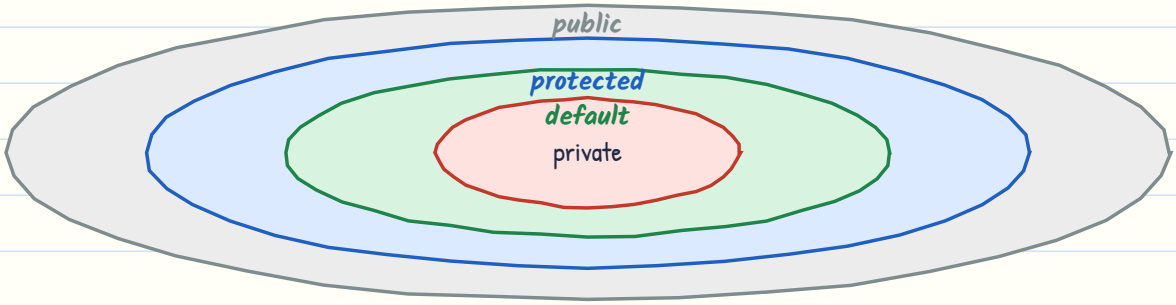


Fig 4: Visibility circles

Accessing private members (getters & setters)

Make fields `private` and give controlled access through public methods - this is *encapsulation* in action.

Account.java

```
class Account {
    private double balance;                 // hidden
    public double getBalance() { return balance; }        // getter
    public void setBalance(double b) {                    // setter
        if (b >= 0) balance = b;            // validation!
    }
}
class Test {
    public static void main(String[] a) {
        Account ac = new Account();
        ac.setBalance(5000);
        // ac.balance = 10;   ERROR: balance has private access
        System.out.println(ac.getBalance());
    }
}
```

```
> OUTPUT
5000.0
```

3. Constructors

A ***constructor*** is a special method that runs automatically when an object is created, used to initialise fields.

- Same ***name as the class***; has ***no return type*** (not even void).
- Called automatically by **new**.
- If you write none, Java gives a hidden ***default constructor***.
- Cannot be **static**, **final** or **abstract**.

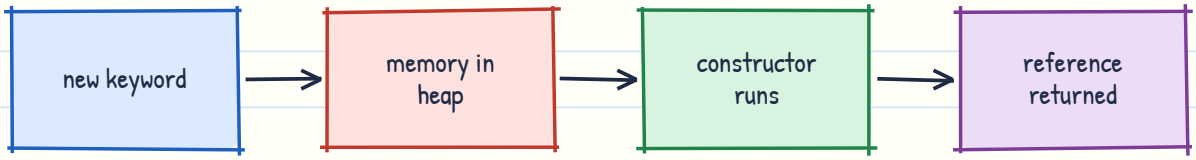


Fig 5: What happens at new Student("Ravi", 21)

```java
class Student {
    String name; int age;
    Student() { name = "Unknown"; age = 0; }          // default (no-arg)
    Student(String n, int a) { name = n; age = a; }   // parameterized
    Student(Student s) { name = s.name; age = s.age; } // copy
}
```

Overloaded constructors

Several constructors with the ***same name but different parameter lists*** – Java picks the one matching the arguments.

```java
Student a = new Student();                 // calls 1st
Student b = new Student("Ravi", 21);       // calls 2nd
Student c = new Student(b);                // calls 3rd
```

Keyword this

`this` is a reference to the ***current object***. Uses: (1) separate field from parameter with same name, (2) call another constructor `this(...)`, (3) pass current object.

```java
class Box {
    int w, h;
    Box(int w, int h) { this.w = w; this.h = h; }   // this.w = field
    Box(int s)        { this(s, s); }               // calls Box(int,int)
}
```

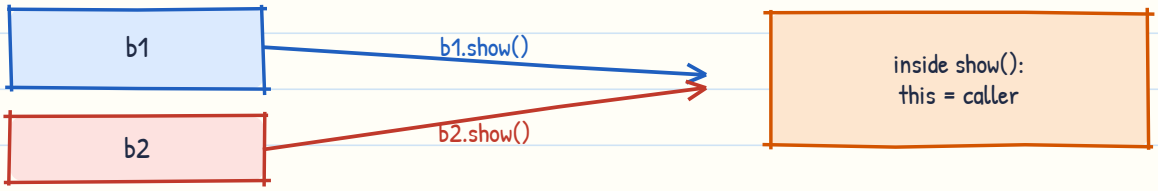


Fig 6: this points to whichever object is calling

4. Nested Classes & final

A class declared ***inside another class*** is a nested class. It helps group classes used only in one place.

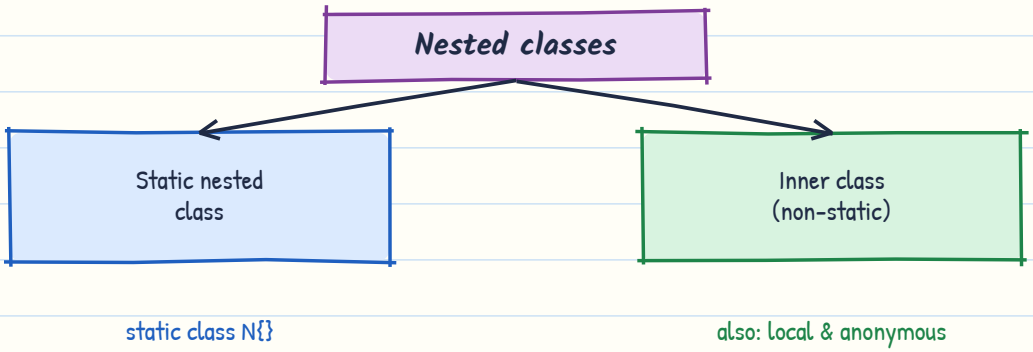


Fig 7: Types of nested classes

```
class Outer {
    int x = 10;
    class Inner {                          // inner class
        void show() { System.out.println("x = " + x); }  // sees Outer's members
    }
    static class SNested { void hi() { System.out.println("static nested"); } }
    public static void main(String[] a) {
        Outer o = new Outer();
        Outer.Inner in = o.new Inner();          // needs outer object
        in.show();
        new Outer.SNested().hi();                // no outer object needed
    }
}
```

```
> OUTPUT
x = 10
static nested
```

Final class and final method

Use	Effect	Example
final variable	value cannot change	final int MAX = 5;
final method	cannot be overridden	final void show(){}
final class	cannot be extended	final class Util{} (e.g. String)

Passing arguments: by value / by reference

Java is ***always pass-by-value***. For primitives the *value* is copied; for objects the *reference* (address) is copied - so the method can change the object's content but cannot re-point the caller's variable.

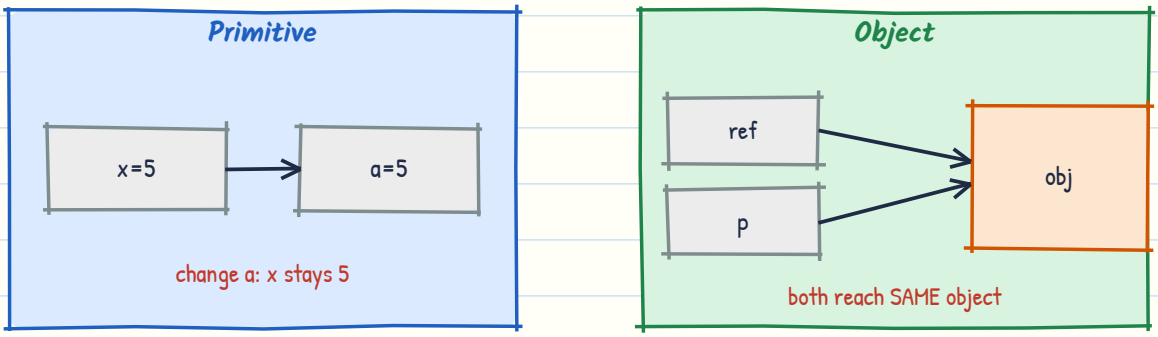


Fig 8: primitive copy vs object reference copy

```
static void change(int a)     { a = 50; }          // original unchanged
static void change(Box b)     { b.w = 50; }        // original CHANGED
int x = 5;  change(x);        System.out.println(x);      // 5
Box bx = new Box(1,1);  change(bx);  System.out.println(bx.w); // 50
```

```
> OUTPUT
5
50
```

5. Methods

A *method* is a named block of code that performs one task and can be reused. It reduces repetition and makes programs modular.

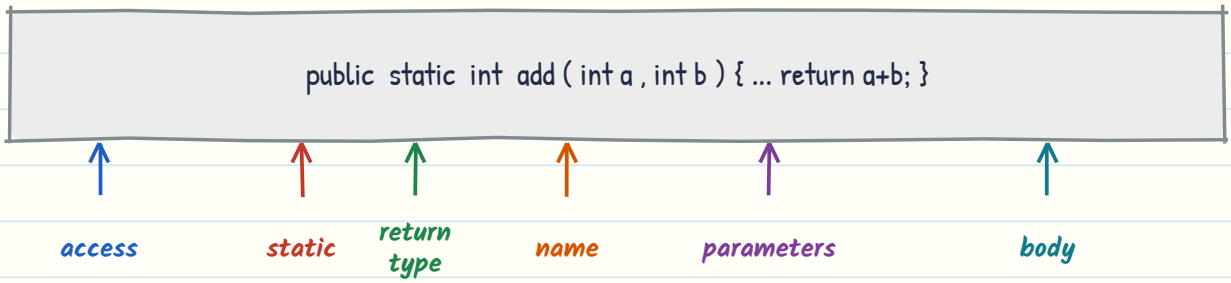


Fig 9: Anatomy of a method

```java
static int add(int a, int b) {     // defining
    return a + b;
}
public static void main(String[] args) {
    int r = add(4, 6);               // calling
    System.out.println(r);
}
```

```
> OUTPUT
10
```

- *Parameters* = variables in the definition; *arguments* = real values passed in the call.
- `void` methods return nothing; others must `return` a matching value.

Method overloading (compile-time polymorphism)

Same method name in one class with *different parameter lists* (number, type or order). Return type alone is NOT enough.

```java
int    add(int a, int b)          { return a + b; }
double add(double a, double b)    { return a + b; }
int    add(int a, int b, int c)   { return a + b + c; }
// add(2,3) -> 1st   add(2.5,3.5) -> 2nd   add(1,2,3) -> 3rd
```

Objects as parameters & returning objects

```java
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    Point add(Point p) {                       // object as parameter
        return new Point(x + p.x, y + p.y);    // object returned
    }
}
Point r = new Point(1,2).add(new Point(3,4));  // r = (4,6)
```

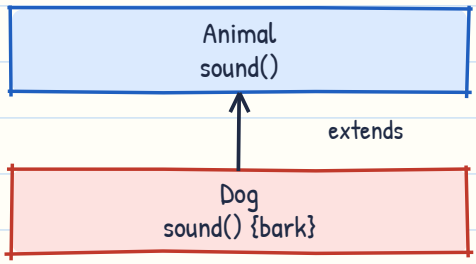
Access control for methods & nesting

Methods use the same modifiers as fields. *Nesting of methods:* Java does not allow a method *inside* a method, but one method may *call* another.

```java
static int square(int n) { return n * n; }
static int sumSquares(int a, int b) { return square(a) + square(b); }  // nested call
// sumSquares(3,4) = 9 + 16 = 25
```

Method overriding

A subclass redefines a method of its parent with the **same name, parameters and return type** (runtime polymorphism). Use `@Override`.



Animal a = new Dog();

a.sound() -> "Woof"

child version runs!

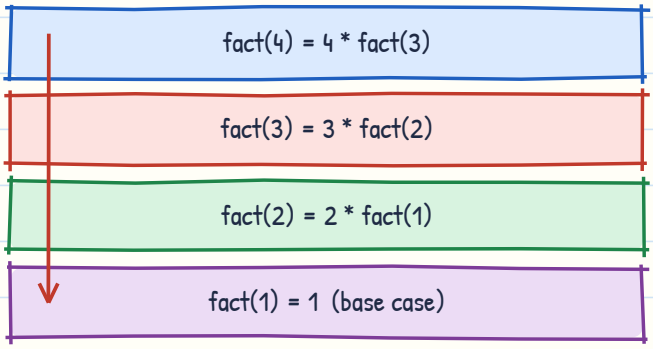
Fig 10: Overriding

	Overloading	Overriding
Where	same class	parent + child
Signature	must differ	must be same
Binding	compile time	run time
Purpose	many forms of input	change inherited behaviour

Recursive methods

A method that **calls itself**. It needs (1) a **base case** to stop and (2) a **recursive case** that moves towards it.

```
static int fact(int n) {
    if (n <= 1) return 1;          // base case
    return n * fact(n - 1);        // recursive case
}
// fact(4) = 24
```



returns 24

returns 6

returns 2

returns 1

calls go down,

answers return up

Fig 11: Call stack of fact(4)

> **Warning**
> No base case = infinite recursion = StackOverflowError. Always ask: "where does it stop?"

Static and final attributes - recap

Keyword	On variable	On method
static	one shared copy per class	call without object; no this
final	constant value	cannot be overridden

Quick Revision & Exam Questions

Topic	Memory line
Object	new -> heap; variable holds reference
s2 = s1	copies reference, one object only
Constructor	class name, no return type, auto-called
this	current object; this(...) calls another constructor
Pass by	Java is always pass-by-value (refs copied)
Overload / Override	compile-time / runtime polymorphism
Recursion	base case + recursive case

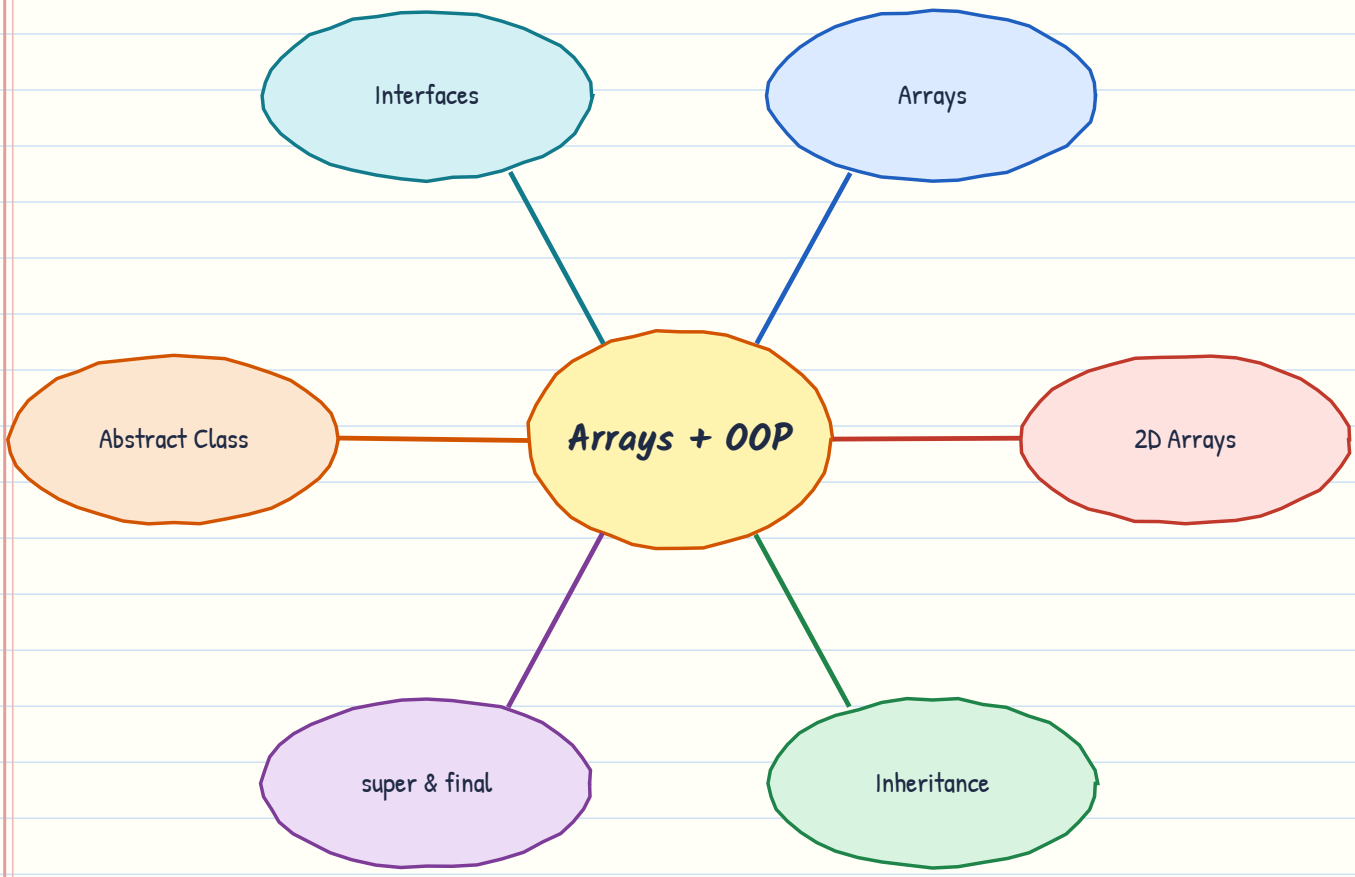
Likely exam questions

- Define class and object; write a program creating objects of a class.
- Explain access modifiers with a table and example.
- What is a constructor? Explain types and overloaded constructors.
- Explain keyword this with examples.
- Differentiate method overloading and overriding.
- Explain pass-by-value and pass-by-reference in Java.
- Write a recursive program for factorial / Fibonacci.
- Write a note on nested and inner classes.

UNIT III

Arrays, Inheritance, Interfaces

1D/2D arrays | Sorting & searching | extends | super | abstract | interface



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. Arrays

An ***array*** stores many values of the ***same type*** in one variable, in ***contiguous memory***, accessed by an index that starts at ***0***. Its size is fixed once created.

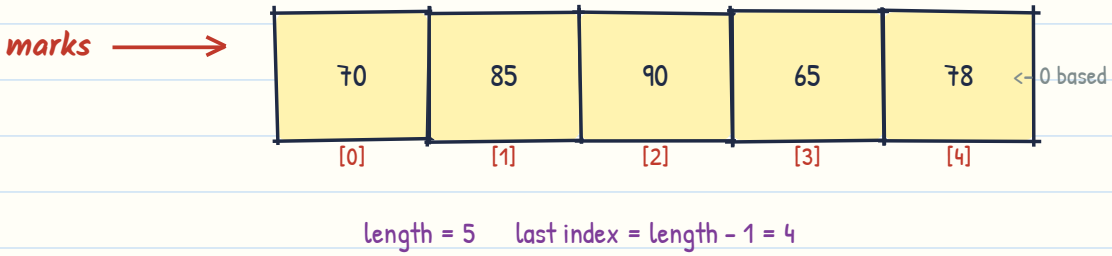


Fig 1: int[] marks = {70, 85, 90, 65, 78}; (stored in heap)

Declaration & initialization

```
int[] a;                        // declare (preferred style)
a = new int[5];                 // create: 5 ints, all 0
int[] b = {10, 20, 30};         // declare + initialize
int c[] = new int[]{1, 2, 3};   // another style
String[] names = new String[3];// default null
```

Array type	Default value
int, long, short, byte	0
double, float	0.0
boolean	false
char	\u0000 (null char)
String / objects	null

Accessing & operations

```java
int[] a = {5, 2, 9, 1};
a[0] = 50;                                  // change element
System.out.println(a.length);               // 4  (length is a field)
int sum = 0, max = a[0];
for (int i = 0; i < a.length; i++) {        // traverse
    sum += a[i];
    if (a[i] > max) max = a[i];
}
for (int x : a) System.out.print(x + " ");  // for-each
```

Careful!

Using index = length (or -1) throws ArrayIndexOutOfBoundsException at run time.

Assigning an array to another

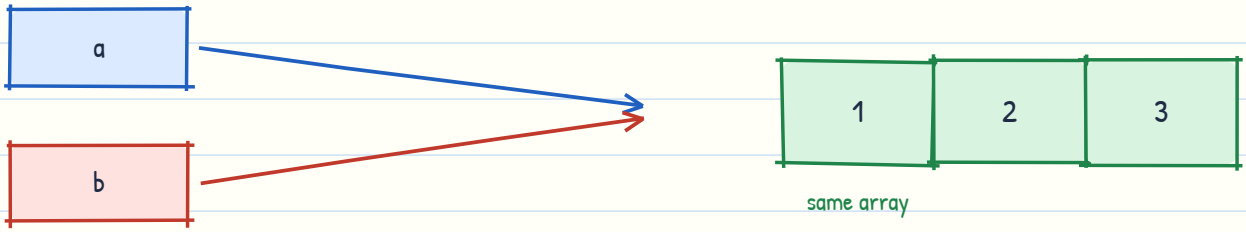


Fig 2: b = a copies the reference; clone() copies the data

```java
int[] a = {1, 2, 3};
int[] b = a;           // alias - change b[0] and a[0] changes
int[] c = a.clone();   // independent copy
int[] d = Arrays.copyOf(a, 5);   // copy + resize -> {1,2,3,0,0}
```

Dynamic change of array size

Arrays cannot grow. To "resize", make a ***bigger new array*** and copy the old values (this is what `ArrayList` does for you).

```java
int[] a = {1, 2, 3};
int[] big = new int[a.length * 2];
System.arraycopy(a, 0, big, 0, a.length);
a = big;               // a now has 6 slots
```

Sorting & searching

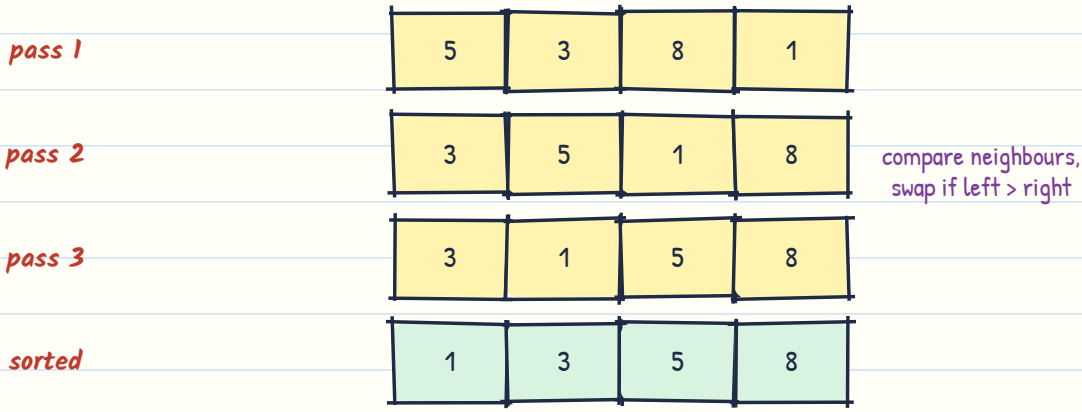


Fig 3: Bubble sort – biggest value "bubbles" to the end

Bubble sort

```
for (int i = 0; i < n - 1; i++)            // passes
    for (int j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) {              // swap
            int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
        }
```

Binary search

```
int low = 0, high = a.length - 1;          // Binary search (array must be SORTED)
while (low <= high) {
    int mid = (low + high) / 2;
    if (a[mid] == key)      return mid;
    else if (a[mid] < key)  low = mid + 1;
    else                    high = mid - 1;
}
return -1;                                  // not found
```

Class Arrays (java.util.Arrays)

Method	Purpose
Arrays.sort(a)	sort ascending
Arrays.binarySearch(a,key)	search in sorted array
Arrays.toString(a)	print as [1, 2, 3]
Arrays.fill(a, v)	fill with value
Arrays.equals(a, b)	compare contents
Arrays.copyOf(a, len)	copy / resize

Two-dimensional arrays

An array of arrays - think of a *table* with rows and columns: `a[row][col]`.

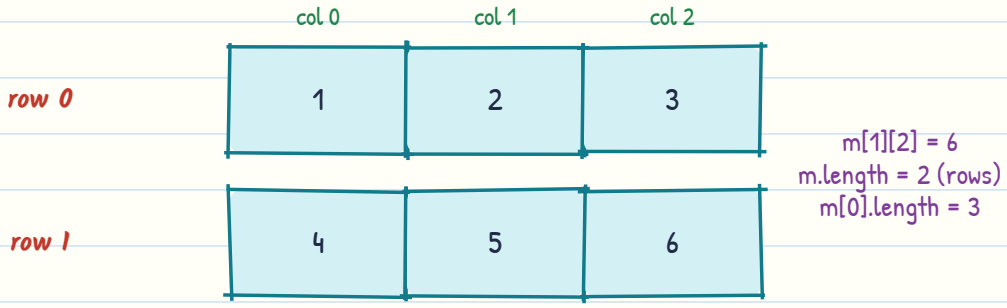


Fig 4: int[][] m = { {1,2,3}, {4,5,6} };

```
int[][] m = new int[2][3];
int[][] t = { {1,2,3}, {4,5,6} };
for (int i = 0; i < t.length; i++) {
    for (int j = 0; j < t[i].length; j++)
        System.out.print(t[i][j] + " ");
    System.out.println();
}
```

```
> OUTPUT
1 2 3
4 5 6
```

Arrays of varying lengths (jagged) & 3D arrays

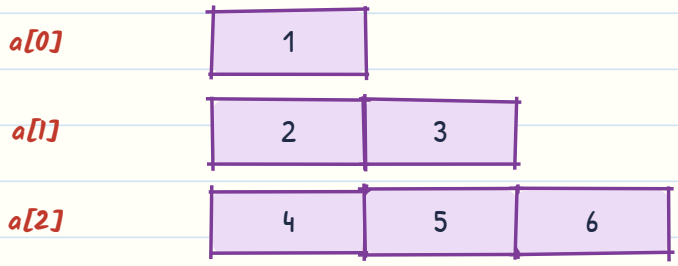


Fig 5: Jagged array - each row has its own length

```
int[][] j = new int[3][];        // only rows created
j[0] = new int[1];  j[1] = new int[2];  j[2] = new int[3];
int[][][] cube = new int[2][3][4];   // 3D: 2 blocks, 3 rows, 4 cols
```

Arrays as vectors: a 1-D array is a vector; for growable vectors use `java.util.Vector` or `ArrayList` (`add()`, `get()`, `remove()`, `size()`).

2. Inheritance

Inheritance lets a class (***subclass / child***) acquire fields and methods of another (***superclass / parent***) using **extends**. It gives ***code reuse*** and an ***IS-A*** relationship.

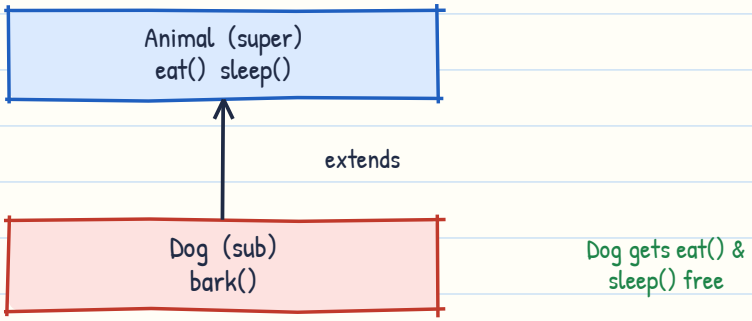


Fig 6: Dog IS-A Animal

```java
class Animal {
    void eat() { System.out.println("eating"); }
}
class Dog extends Animal {
    void bark() { System.out.println("barking"); }
}
public class Test {
    public static void main(String[] a) {
        Dog d = new Dog();
        d.eat();  d.bark();       // eat() inherited
    }
}
```

```
> OUTPUT
eating
barking
```

Types of inheritance

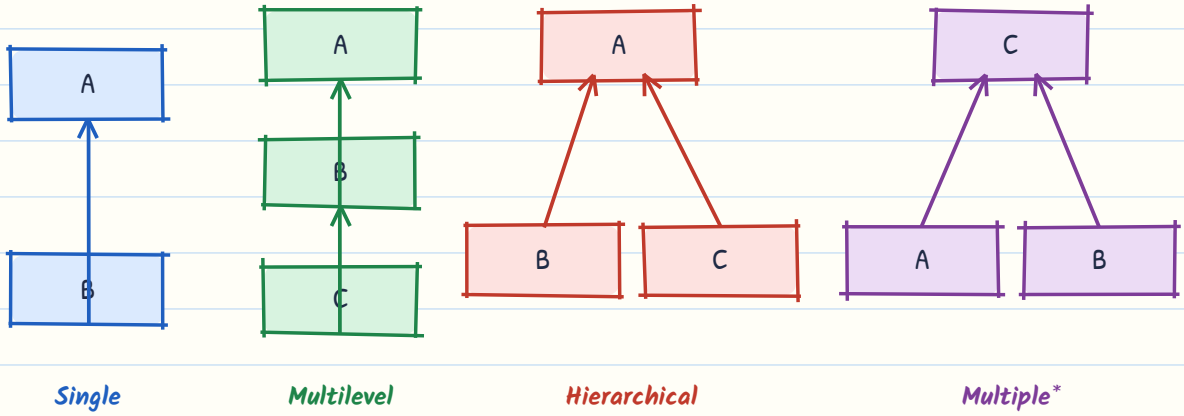


Fig 7: Types of inheritance

> **Important**
>
> *Java does NOT allow multiple inheritance with classes (diamond problem / ambiguity). It is achieved using INTERFACES. Hybrid = combination, also via interfaces.

Universal superclass - Object class

Every Java class ***automatically extends java.lang.Object*** - it is the root of the class tree.

So all objects have these methods:

Method	Purpose
toString()	text form of object
equals(obj)	compare objects
hashCode()	integer hash
getClass()	runtime class info

Multilevel inheritance & constructors

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }
new C();     // constructors run TOP -> DOWN
```

```
> OUTPUT
A
B
C
```

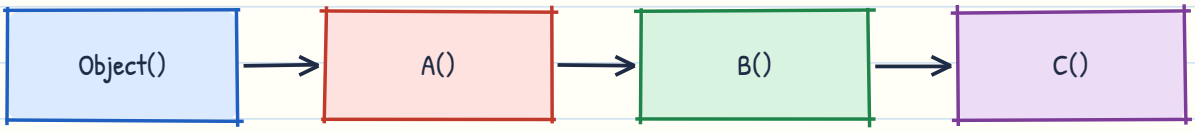


Fig 8: Constructor order parent first, child last

Keyword super

- `super.member` - access parent's field/method hidden by child.
- `super(args)` - call parent constructor; must be the *first* statement.
- Without it Java inserts `super();` automatically.

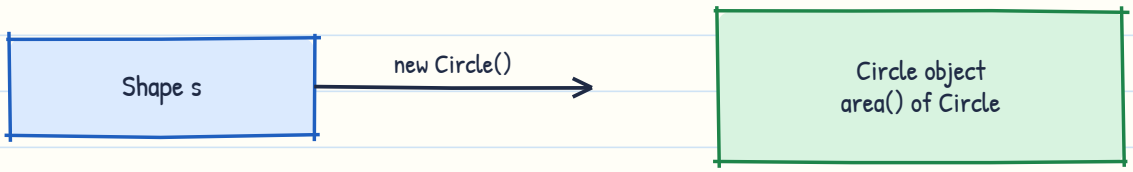
```
class Person { String name; Person(String n) { name = n; }
               void show() { System.out.println("Person " + name); } }
class Student extends Person {
    int roll;
    Student(String n, int r) { super(n); roll = r; }   // parent constructor
    void show() { super.show(); System.out.println("Roll " + roll); }
}
```

Final, access control & inheritance

- `final class` cannot be extended; `final` method cannot be overridden.
- `private` members are NOT inherited (use getters); `protected` & `public` are.
- Subclass can *widen* but not narrow access when overriding.

Dynamic method dispatch (runtime polymorphism)

A *parent reference can hold a child object*; the method that runs is decided at *run time* by the actual object.



s.area() -> JVM looks at the OBJECT, runs Circle.area()

Fig 9: Parent ref -> child object

```
class Shape  { void draw() { System.out.println("shape"); } }
class Circle extends Shape { void draw() { System.out.println("circle"); } }
class Square extends Shape { void draw() { System.out.println("square"); } }
Shape s;
s = new Circle();  s.draw();     // circle
s = new Square();  s.draw();     // square
```

Abstract classes

An ***abstract class*** is incomplete – it may have abstract methods (no body) that children ***must*** implement. You cannot create its object.

```
abstract class Shape {
    abstract double area();                  // no body
    void info() { System.out.println("I am a shape"); }   // normal method OK
}
class Rect extends Shape {
    double w, h;  Rect(double w, double h) { this.w = w; this.h = h; }
    double area() { return w * h; }          // must implement
}
// Shape s = new Shape();  ERROR      Shape s = new Rect(2,3);  OK
```

3. Interfaces

An ***interface*** is a 100% abstract contract: it lists *what* a class must do, not *how*. A class ***implements*** it. This gives ***multiple inheritance*** and loose coupling.

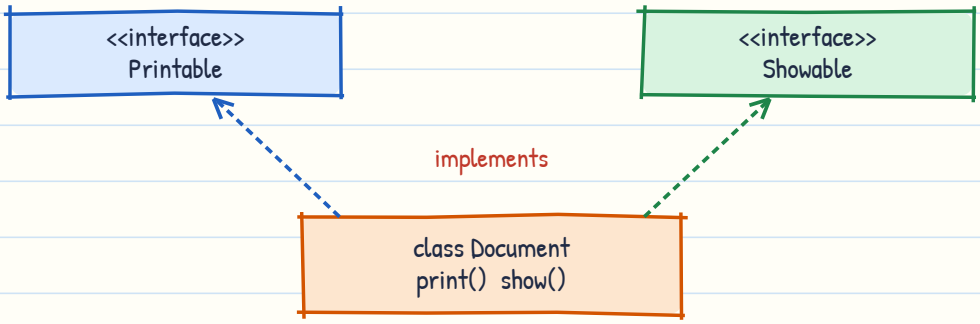


Fig 10: class implements two interfaces (multiple inheritance of type)

```java
interface Printable {                  // declaration
    int MAX = 10;                      // public static final (constant)
    void print();                      // public abstract
}
interface Showable { void show(); }
class Doc implements Printable, Showable {      // multiple interfaces
    public void print() { System.out.println("printing"); }  // must be public
    public void show()  { System.out.println("showing"); }
}
```

	Abstract class	Interface
Keyword	extends (one only)	implements (many)
Methods	abstract + normal	abstract (+default/static)
Variables	any kind	only constants
Constructor	yes	no

Nested interfaces & inheritance of interfaces

```java
interface A { void a(); }
interface B extends A { void b(); }       // interface extends interface
class C implements B { public void a(){}  public void b(){} }  // must do both
class Outer { interface Inner { void f(); } }   // nested interface
```

Default & static methods (Java 8)

```java
interface Vehicle {
    void run();
    default void honk() { System.out.println("beep"); }   // has a body
    static void rules() { System.out.println("drive safe"); }
}
Vehicle.rules();     // call static method with interface name
```

Functional interfaces & annotations

A *functional interface* has exactly *one abstract method*; it can be written as a short *lambda* expression. `@FunctionalInterface` makes the compiler check it.

```java
@FunctionalInterface
interface Calc { int op(int a, int b); }
Calc add = (a, b) -> a + b;          // lambda
Calc mul = (a, b) -> a * b;
System.out.println(add.op(3, 4));    // 7
```

```
> OUTPUT
7
```

Annotation	Meaning
@Override	method overrides parent - compiler checks
@FunctionalInterface	exactly one abstract method
@Deprecated	old, avoid using
@SuppressWarnings	hide compiler warnings

Quick Revision & Exam Questions

Topic	Memory line
Array	same type, fixed size, index from 0, length field
a = b	alias; use clone() / copyOf() to copy
extends	one parent only for classes
super	parent member / super(...) first line
Dispatch	parent ref, child object, run-time choice
abstract	cannot instantiate; abstract methods forced
interface	all abstract, implements many, default/static allowed

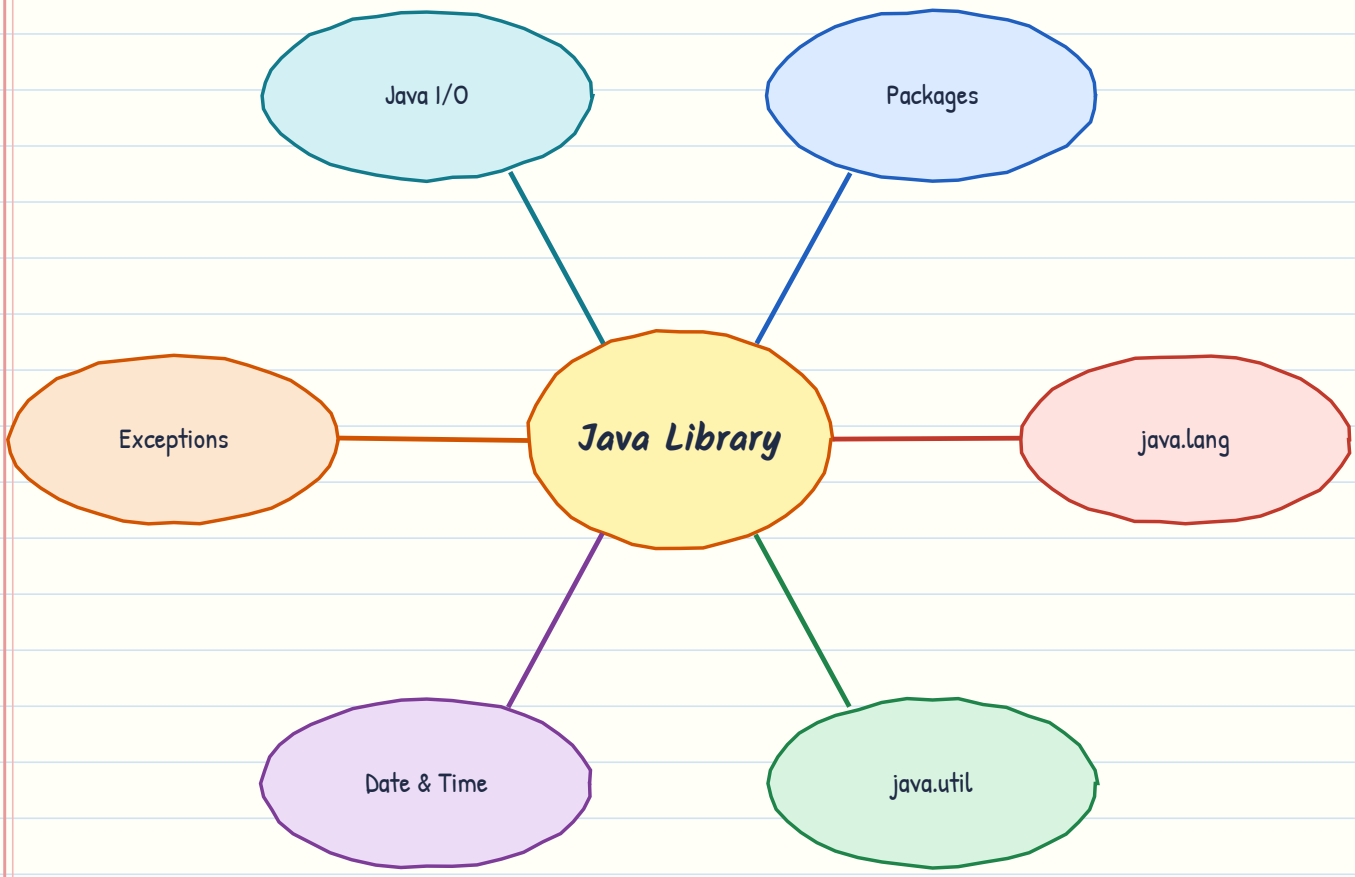
Likely exam questions

- Explain declaration, initialization and traversal of arrays with a program.
- Write a program for bubble sort / binary search.
- Explain 2-D and jagged arrays with diagrams.
- Explain types of inheritance with examples; why no multiple inheritance?
- Explain keyword super and constructor order in inheritance.
- What is dynamic method dispatch? Explain with a program.
- Differentiate abstract class and interface.
- Explain default, static and functional interfaces with examples.

UNIT IV

Packages & Java Library, Exceptions, I/O

Packages | java.lang | java.util | java.time | try-catch | Streams



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. Packages

A **package** is a folder-like container that groups related classes and interfaces. It avoids name clashes, controls access and keeps big projects organised.

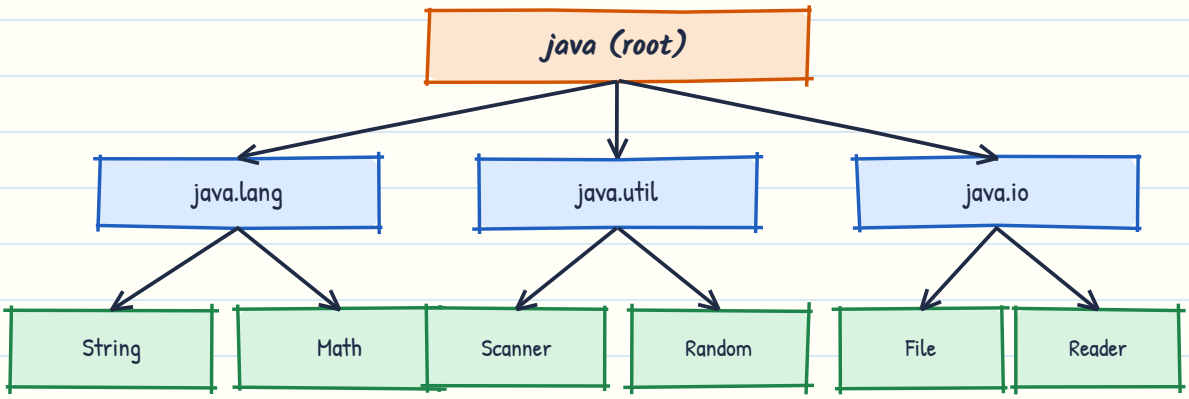


Fig 1: Package = folder -> classes inside

Defining & importing packages

mypack/Msg.java

```
package mypack;                  // MUST be the first line
public class Msg {
    public void show() { System.out.println("Hello from package"); }
}
```

Test.java

```
import mypack.Msg;               // one class
// import mypack.*;               // all classes of package
// import static java.lang.Math.*;   // static members
class Test { public static void main(String[] a) { new Msg().show(); } }
```

```
javac -d . Msg.java       // -d . creates folder mypack/
javac Test.java
java Test
```

- **PATH** = where the OS finds `javac`/`java` tools.
- **CLASSPATH** = where the JVM finds your `.class` files / packages (`-cp` option).
- `java.lang` is imported automatically; all others need `import`.

Access	Same pkg	Other pkg
public	yes	yes
protected	yes	only subclass
default	yes	no
private	no (class only)	no

Packages in Java SE

Package	Contains
java.lang	core: String, Math, Object, wrappers, Thread
java.util	collections, Scanner, Random, Date
java.io	input / output streams, File
java.time	modern date & time API
java.sql	JDBC database classes
javafx.*	GUI classes

2. java.lang Package

- ***Object*** - root class (toString, equals, hashCode, getClass).
- ***String / StringBuffer***, ***Math***, ***Wrapper classes***, ***System***, ***Thread***, ***Exception classes***.

class Math

Method	Result	Method	Result
abs(-5)	5	pow(2,3)	8.0
sqrt(25)	5.0	max(4,9)	9
ceil(4.2)	5.0	floor(4.8)	4.0
round(4.5)	5	random()	0.0 to <1.0

Enumeration (enum)

An ***enum*** is a fixed set of named constants - safer than plain ints.

```
enum Day { MON, TUE, WED }
public class E {
    public static void main(String[] a) {
        Day d = Day.TUE;
        switch (d) { case TUE: System.out.println("Tuesday"); break; default: break; }
        for (Day x : Day.values()) System.out.print(x + " ");   // all constants
        System.out.println(d.ordinal());   // position = 1
    }
}
```

```
> OUTPUT
Tuesday
MON TUE WED 1
```

Wrapper classes, autoboxing & unboxing

A **wrapper class** turns a primitive into an object (needed for collections).

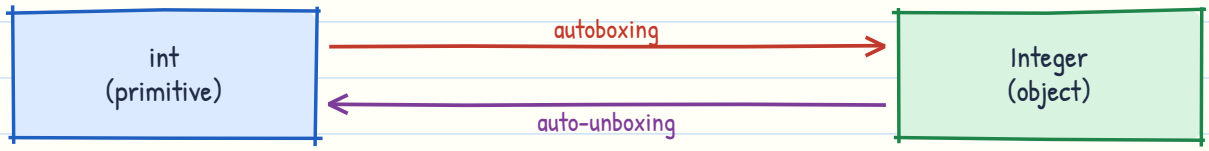


Fig 2: Boxing and unboxing

Primitive	Wrapper	Primitive	Wrapper
byte	Byte	long	Long
short	Short	float	Float
int	Integer	double	Double
char	Character	boolean	Boolean

```
Integer a = 10;                  // autoboxing (int -> Integer)
int b = a;                       // auto-unboxing
int n = Integer.parseInt("123"); // String -> int
String s = Integer.toString(45); // int -> String
System.out.println(Integer.MAX_VALUE);   // 2147483647
```

3. java.util Classes & Interfaces

Class	Use
Scanner	read input
Random	random numbers
Formatter	formatted output (like printf)
ArrayList, HashMap, ...	collections framework

```java
Random r = new Random();
int x = r.nextInt(100);          // 0 to 99
double y = r.nextDouble();       // 0.0 to 1.0
Formatter f = new Formatter();
f.format("%s scored %d", "Ravi", 90);
System.out.println(f);           // Ravi scored 90
```

Time package (java.time)

Class	Represents
LocalDate	date only (2026-10-17)
LocalTime	time only (14:30:05)
LocalDateTime	date + time
Instant	a moment on the time line (UTC)
DateTimeFormatter	format / parse text
TemporalAdjusters	shortcuts like "next Monday"

```java
LocalDate today = LocalDate.now();
Instant now = Instant.now();                        // java.time.Instant
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd-MM-yyyy");
System.out.println(today.format(f));
LocalDate nm = today.with(TemporalAdjusters.next(DayOfWeek.MONDAY));
LocalDate last = today.with(TemporalAdjusters.lastDayOfMonth());
```

Pattern tip

Pattern letters: dd=day, MM=month, yyyy=year, HH=hour(24), mm=minute, ss=second.

(Capital M = month, small m = minute!)

4. Exception Handling

An ***exception*** is an unwanted event at ***run time*** that disturbs normal flow (divide by zero, file missing). Handling it prevents the program from crashing.

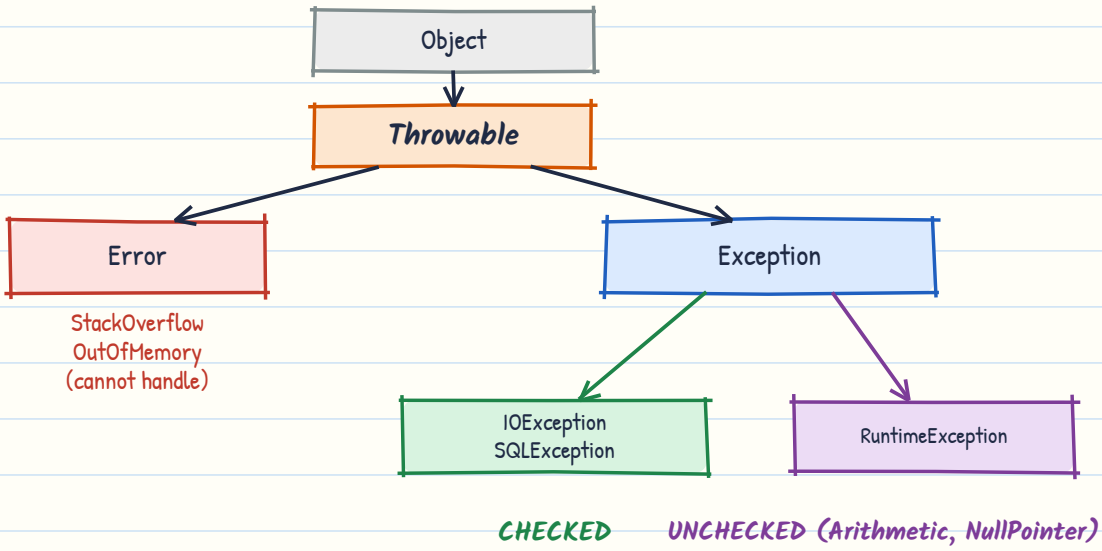


Fig 3: Exception class hierarchy

	Checked	Unchecked
Checked by	compiler (must handle/declare)	not checked at compile time
Examples	IOException, SQLException	ArithmeticException, NullPointerException
Parent	Exception	RuntimeException

try, catch, finally

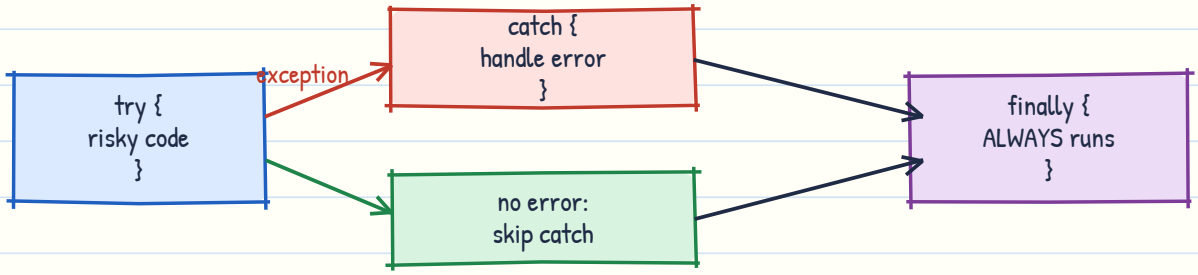


Fig 4: Flow of try-catch-finally

```
try {
    int r = 10 / 0;                    // throws ArithmeticException
    System.out.println(r);             // skipped
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("cleanup always runs");
}
```

```
> OUTPUT
Error: / by zero
cleanup always runs
```

Multiple catch clauses

```
try {
    int[] a = new int[3];  a[5] = 1;
    String s = null;       s.length();
} catch (ArrayIndexOutOfBoundsException e) { System.out.println("bad index"); }
  catch (NullPointerException e)           { System.out.println("null!"); }
  catch (Exception e)                      { System.out.println("other"); }  // LAST
```

Rule

Order matters: put specific (child) exceptions first and general Exception last, otherwise compile error (unreachable catch).

throw and throws

Keyword	Meaning
throw	actually raises an exception object (inside method)
throws	declares in method header that it MAY raise it

```
static void check(int age) throws Exception {      // declares
    if (age < 18) throw new Exception("Not eligible");   // raises
    System.out.println("Eligible");
}
public static void main(String[] a) {
    try { check(15); } catch (Exception e) { System.out.println(e.getMessage()); }
}
```

```
> OUTPUT
Not eligible
```

Class Throwable & user-defined exceptions

Useful methods of *Throwable*: `getMessage()`, `printStackTrace()`, `toString()`.

Create your own exception by extending `Exception`.

```java
class MyEx extends Exception {
    MyEx(String msg) { super(msg); }
}
// throw new MyEx("custom problem");
```

5. Java I/O and Files

Java reads/writes data using ***streams*** – a flow of data between a program and a source/destination (keyboard, file, network).



Fig 5: Input stream & output stream

Standard I/O streams

Stream	Meaning	Object
System.in	input (keyboard)	InputStream
System.out	normal output (screen)	PrintStream
System.err	error output	PrintStream

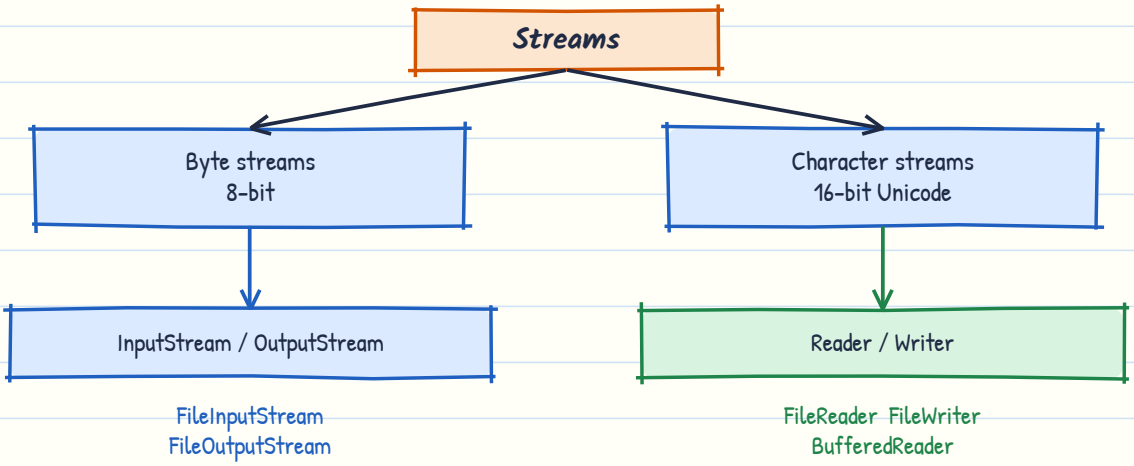


Fig 6: Types of streams

- ***Byte streams*** – images, audio, any binary data.
- ***Character streams*** – text files; handle Unicode & line reading.

Reading and writing files

```
import java.io.*;
public class FileDemo {
    public static void main(String[] a) throws IOException {
        FileWriter w = new FileWriter("data.txt");      // write
        w.write("Hello File");  w.close();
        FileReader r = new FileReader("data.txt");      // read
        int ch;
        while ((ch = r.read()) != -1) System.out.print((char) ch);
        r.close();
    }
}
```

```
> OUTPUT
Hello File
```

```
Scanner sc = new Scanner(new File("data.txt"));   // easy line reading
while (sc.hasNextLine()) System.out.println(sc.nextLine());
File f = new File("data.txt");
f.exists();  f.length();  f.getName();  f.delete();   // File class methods
```

Good practice

Always close streams (or use try-with-resources: try (FileReader r = new FileReader(..)) { }) so data is flushed and resources freed.

Quick Revision & Exam Questions

Topic	Memory line
package	first statement; folders; import to use
Wrapper	primitive <-> object, auto boxing/unboxing
Exception	Throwable -> Error / Exception -> Runtime
finally	always runs (cleanup)
throw / throws	raise / declare
Streams	byte (InputStream) vs character (Reader)

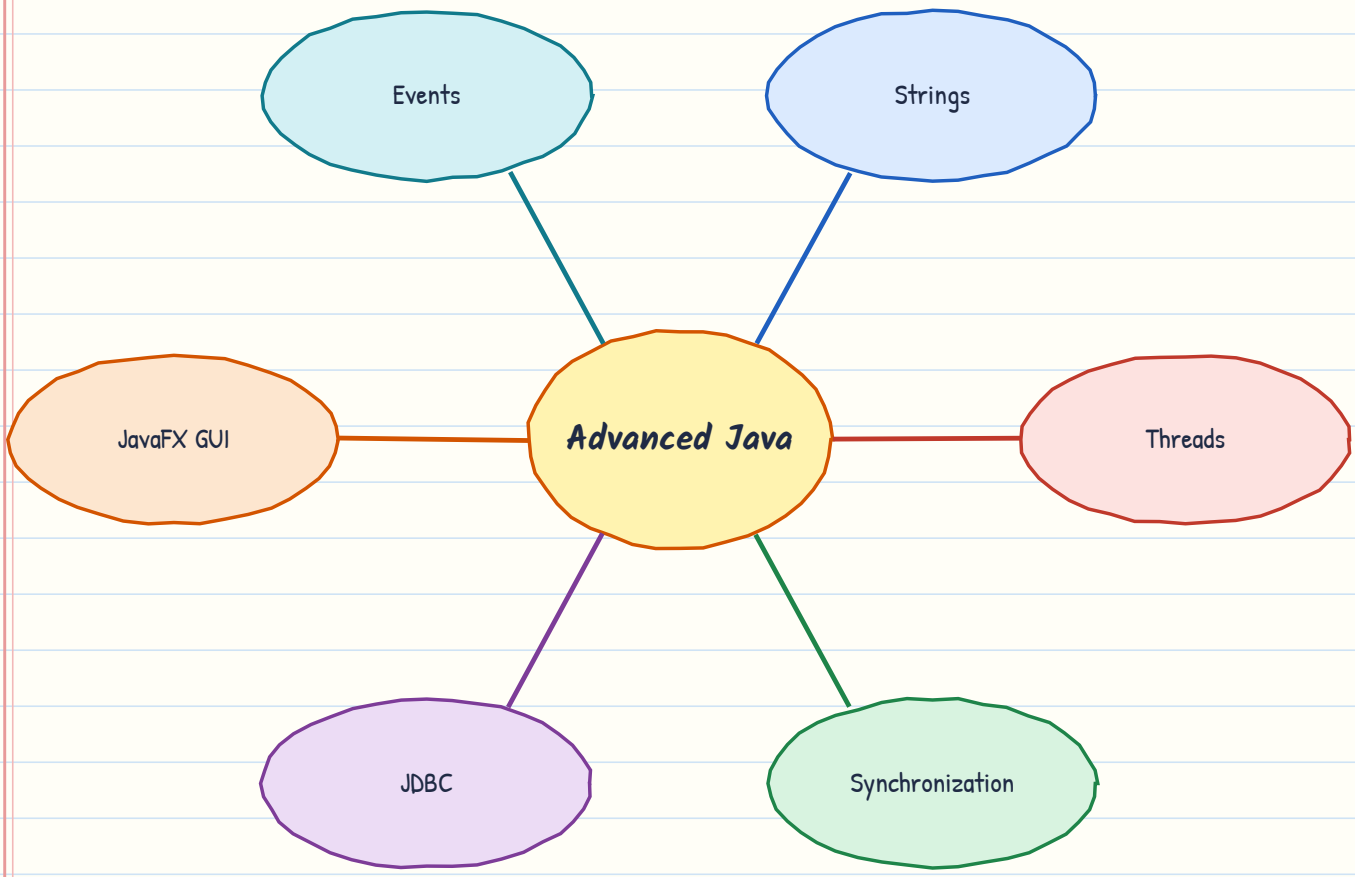
Likely exam questions

- What is a package? How do you create and import one?
- Explain wrapper classes with autoboxing and unboxing.
- Explain Math, Random and Formatter classes.
- Explain java.time classes and date formatting.
- Explain exception hierarchy; checked vs unchecked.
- Explain try, catch, finally, throw and throws with programs.
- Explain byte and character streams; write a file read/write program.

UNIT V

Strings, Multithreading, JDBC, JavaFX

String & StringBuffer | Thread life cycle | Synchronization | JDBC | GUI



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

I. String Handling

A ***String*** is a sequence of characters. In Java it is an ***object*** of class **`String`**, and it is ***immutable*** – once created it can never be changed (every "change" makes a new String).

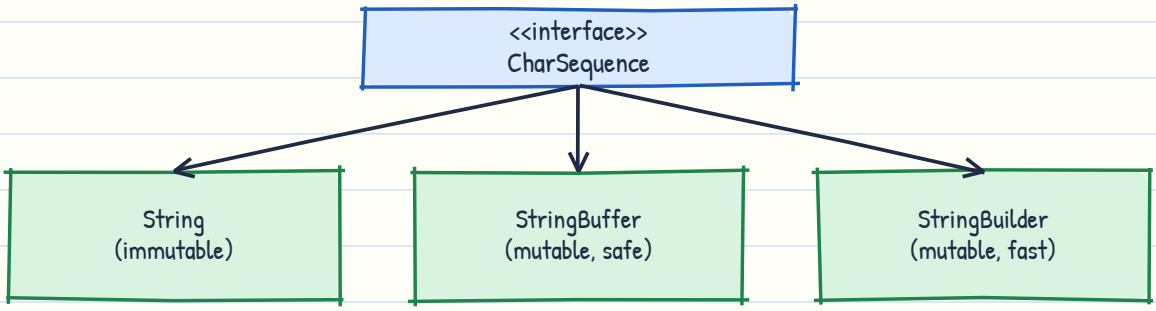


Fig 1: Interface CharSequence and its implementers

Creating strings & the string pool

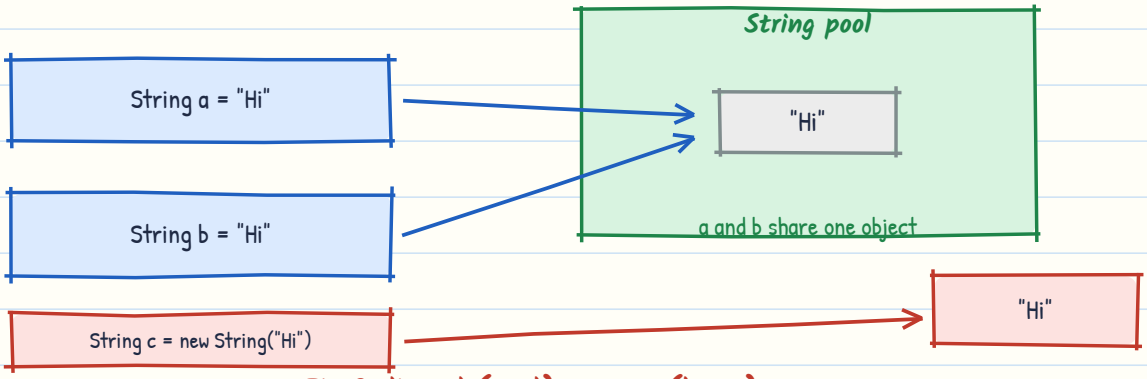


Fig 2: literal (pool) vs new (heap)

```
String s1 = "Java";                 // literal -> pool
String s2 = new String("Java");     // new object in heap
System.out.println(s1 == s2);       // false  (different objects)
System.out.println(s1.equals(s2));  // true   (same content)
```

Classic exam trap

compares REFERENCES, equals() compares CONTENT. Always use equals() for string

Extracting characters

Method	Purpose	Example ("HELLO")
length()	number of chars	5
charAt(i)	char at index	charAt(1) = E
toCharArray()	to char[]	[H,E,L,L,O]
getChars(...)	copy part into char[]	-

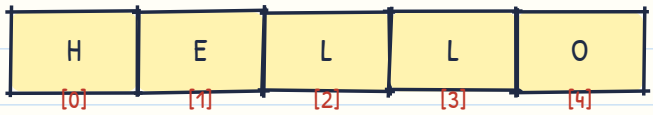


Fig 3: index positions of "HELLO"

Comparison

Method	Meaning
equals(s)	exact content match (case-sensitive)
equalsIgnoreCase(s)	match ignoring case
compareTo(s)	0 if equal, <0 if smaller, >0 if bigger (dictionary order)
startsWith / endsWith	prefix / suffix test

Modifying & searching

```
String s = "  Hello World  ";
s.trim();                          // "Hello World"
s.toUpperCase();  s.toLowerCase();
"Hello".substring(1, 4);           // "ell"   (start incl., end excl.)
"Hello".concat(" Java");           // "Hello Java"
"banana".replace('a', 'o');        // "bonono"
"Hello".indexOf('l');              // 2  (first)
"Hello".lastIndexOf('l');          // 3
"Hello".contains("ell");           // true
"a,b,c".split(",");                // {"a","b","c"}
```

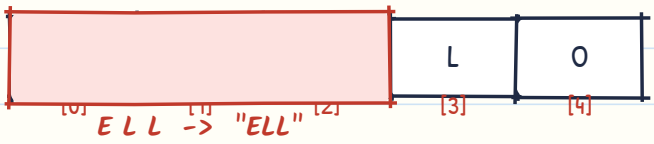


Fig 4: substring(1,4) on "HELLO"

Class StringBuffer (mutable string)

Use when you modify text often - it changes the *same object* instead of creating new ones.

`StringBuilder` is the same but not thread-safe (faster).

```java
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");        // Hello World
sb.insert(0, ">> ");        // >> Hello World
sb.reverse();               // dlroW olleH >>
sb.delete(0, 3);  sb.replace(0, 1, "D");
sb.length();  sb.capacity();  // default capacity 16
```

	String	StringBuffer
Mutable?	No	Yes
Thread-safe	yes (immutable)	yes (synchronized)
Speed (many edits)	slow	fast

2. Multithreaded Programming

A ***thread*** is a lightweight path of execution inside a program. ***Multithreading*** runs many threads at once so work overlaps - e.g. a music player plays audio while the UI stays responsive.

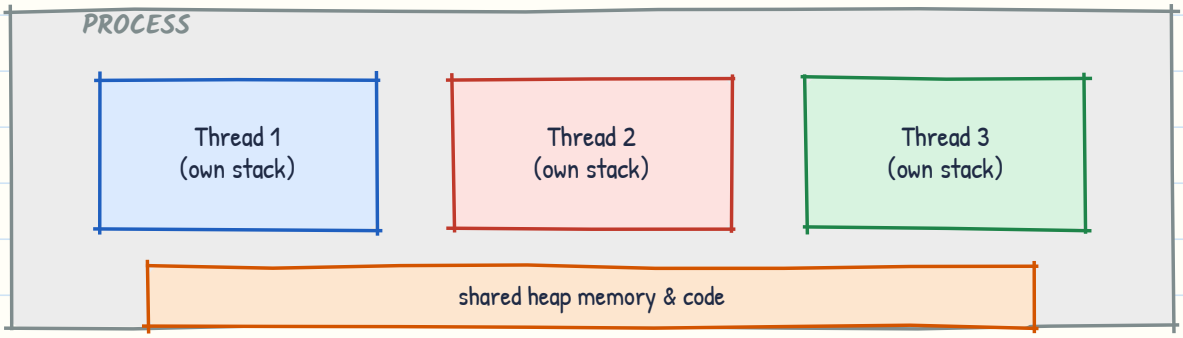


Fig 5: One process, many threads (sharing memory)

- ***Need:*** better CPU use, responsive apps, faster results on ***multi-core*** processors (each core runs a thread truly in parallel).
- ***Process*** = heavy, own memory | ***Thread*** = light, shares memory.

Main thread & creating threads

Every program starts with the ***main thread***. Two ways to create new threads:

```java
class A extends Thread {                        // way 1: extend Thread
    public void run() { System.out.println("Thread A running"); }
}
class B implements Runnable {                   // way 2: implement Runnable (preferred)
    public void run() { System.out.println("Thread B running"); }
}
public class Demo {
    public static void main(String[] args) {
        new A().start();                        // start(), NOT run()!
        new Thread(new B()).start();
        System.out.println(Thread.currentThread().getName());   // main
    }
}
```

Remember

start() creates a NEW thread which calls run(). Calling run() directly is just a normal method call - no new thread!

Thread states (life cycle)

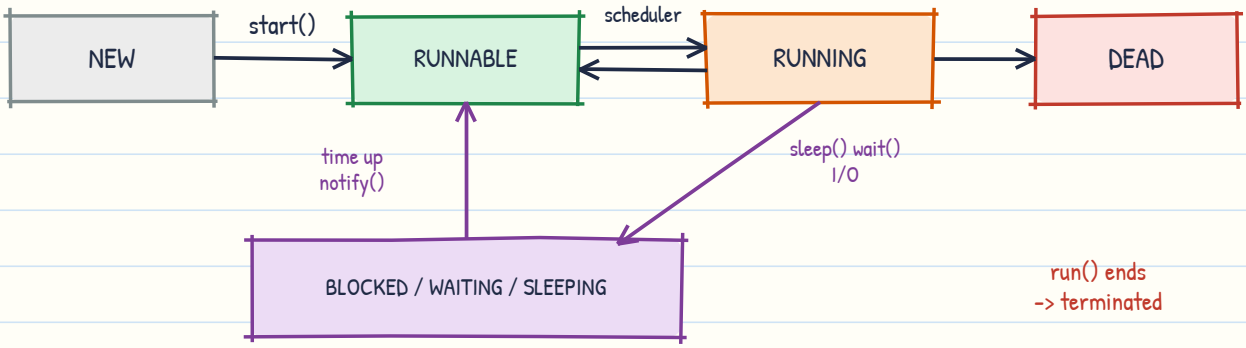


Fig 6: Life cycle of a thread

Thread priority & useful methods

Method	Purpose
start()	begin new thread
sleep(ms)	pause for time
join()	wait for another thread to finish
setPriority(1-10)	hint to scheduler (MIN=1, NORM=5, MAX=10)
isAlive(), getName()	status / name

Race condition, synchronization & deadlock

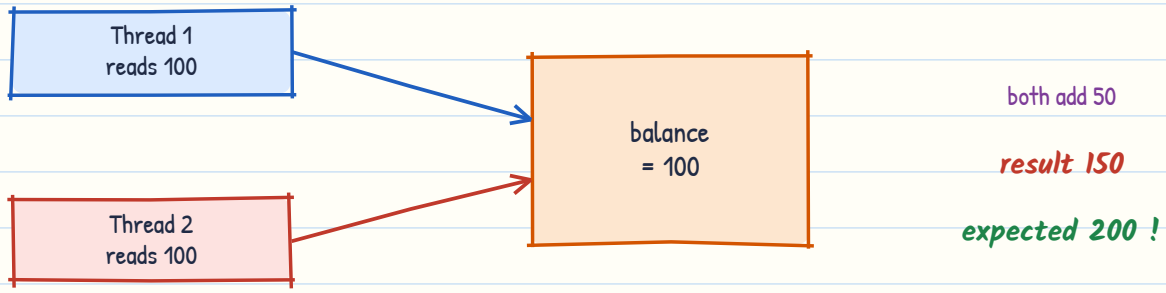
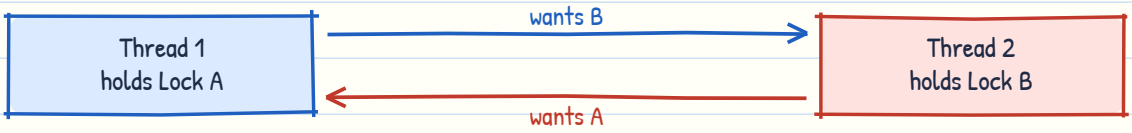


Fig 7: Race condition - two threads update the same data

Synchronization lets only ***one thread at a time*** enter critical code using the `synchronized` keyword (a lock / monitor).

```
class Counter {
    int c = 0;
    synchronized void inc() { c++; }      // one thread at a time
    void other() { synchronized (this) { c--; } }   // block version
}
```



both wait forever = DEADLOCK

Fig 8: Deadlock - each thread waits for the other's lock

Inter-thread communication

Threads cooperate with `wait()`, `notify()`, `notifyAll()` (called inside synchronized code), e.g. ***producer-consumer***: consumer waits till producer has data.



Fig 9: Producer - Consumer

Suspend / Resume / Stop

suspend(), resume(), stop() are DEPRECATED (unsafe, may cause deadlock). Instead use a boolean flag or wait()/notify() to pause, resume and stop threads.

3. Java Database Connectivity (JDBC)

JDBC is the Java API for connecting to a relational database (MySQL) and running SQL from a program.

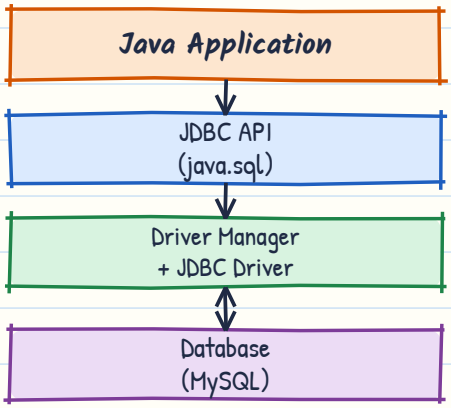


Fig 10: JDBC architecture

Setup steps

- Install MySQL and create a database/table.
- Download **MySQL Connector/J** (`mysql-connector-j.jar`) and add it to the **classpath**.
- Write the program using the 5 JDBC steps below.

Step	Code
1. Load driver	Class.forName("com.mysql.cj.jdbc.Driver")
2. Connect	DriverManager.getConnection(url,user,pass)
3. Statement	con.createStatement()
4. Execute	executeQuery() / executeUpdate()
5. Close	con.close()

```java
import java.sql.*;
public class Db {
    public static void main(String[] a) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "password");
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT id, name FROM student");
        while (rs.next())                                   // move row by row
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        con.close();
    }
}
```

```
> OUTPUT
101 Asha
102 Ravi
```

ResultSet interface

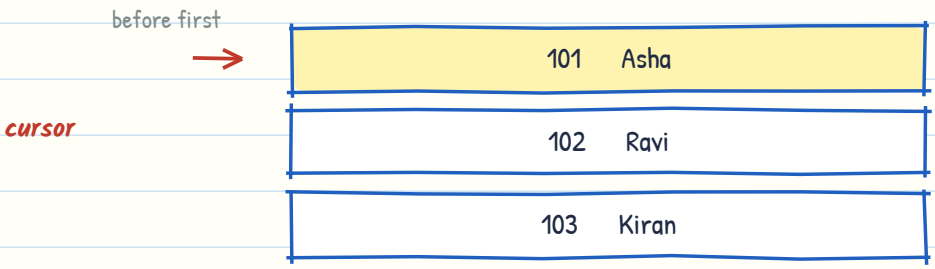


Fig 11: ResultSet cursor moves with next()

Method	Purpose
next()	move to next row; false when no more rows
getInt/getString(col)	read column value
executeUpdate()	INSERT / UPDATE / DELETE, returns rows affected

```java
PreparedStatement ps = con.prepareStatement("INSERT INTO student VALUES(?,?)");
ps.setInt(1, 104);  ps.setString(2, "Meena");
ps.executeUpdate();            // safer than string concatenation (stops SQL injection)
```

4. JavaFX GUI

JavaFX is Java's modern library for desktop GUI apps. **Scene Builder** is a drag-and-drop tool that creates the layout as an **FXML** file.

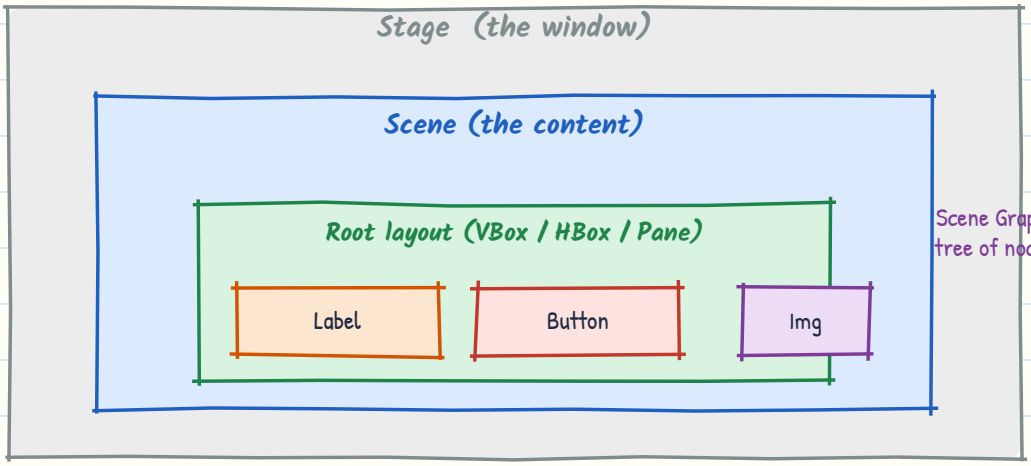


Fig 12: JavaFX app window structure

```
import javafx.application.Application;
import javafx.scene.*;  import javafx.scene.control.*;  import javafx.scene.layout.*;
import javafx.stage.Stage;
public class Hello extends Application {
    public void start(Stage stage) {                    // entry point of JavaFX
        Label lbl = new Label("Hello JavaFX");
        Button btn = new Button("Click me");
        btn.setOnAction(e -> lbl.setText("Button clicked!"));   // event handling
        VBox root = new VBox(10, lbl, btn);                 // layout
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("My App");  stage.show();
    }
    public static void main(String[] a) { launch(a); }
}
```

Displaying text & images

```
Text t = new Text("Welcome");  t.setFont(Font.font("Arial", 24));
ImageView iv = new ImageView(new Image("file:logo.png"));
iv.setFitWidth(100);
```

Laying out nodes in the scene graph

Layout	Arranges nodes
HBox	in a horizontal row
VBox	in a vertical column
BorderPane	top, bottom, left, right, center
GridPane	rows and columns grid

StackPane	on top of each other
FlowPane	wrapping flow

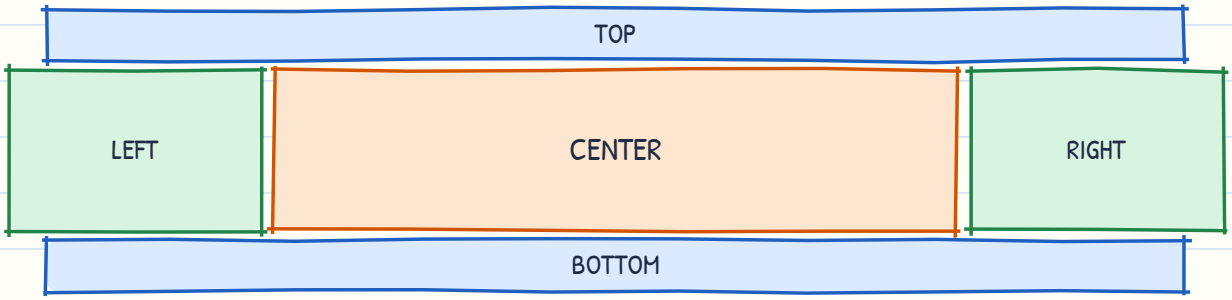


Fig 13: BorderPane regions

Event handling & mouse events

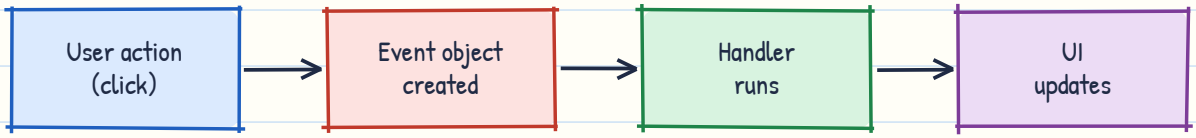


Fig 14: Event flow

```
btn.setOnAction(e -> System.out.println("clicked"));        // button
pane.setOnMouseClicked(e -> System.out.println(e.getX() + "," + e.getY()));
pane.setOnMouseEntered(e -> pane.setStyle("-fx-background-color: yellow;"));
pane.setOnMouseExited(e -> pane.setStyle(""));
```

Mouse event	When
setOnMouseClicked	click
setOnMousePressed / Released	button down / up
setOnMouseMoved / Dragged	cursor moves / drags
setOnMouseEntered / Exited	cursor enters / leaves node

Quick Revision & Exam Questions

Topic	Memory line
String	immutable; == vs equals(); pool
StringBuffer	mutable; append, insert, reverse
Thread create	extends Thread or implements Runnable; start()
States	New - Runnable - Running - Blocked - Dead
synchronized	one thread at a time; prevents race condition
JDBC steps	driver, connect, statement, execute, close

JavaFX	Stage > Scene > layout > nodes

Likely exam questions

- Explain String methods for extraction, comparison, modification and searching.
- Differentiate String and StringBuffer.
- What is multithreading? Explain thread life cycle with a diagram.
- Explain ways of creating threads with programs.
- Explain synchronization, race condition and deadlock.
- Explain JDBC architecture and the steps to connect to MySQL.
- Explain JavaFX window structure; write a program with a button and event handling.